Cairo University
Faculty of Computers and Artificial Intelligence
Artificial Intelligence Department
Information Systems Department

Seed Bank

Quality Intelligence Platform

Graduation Project Final Documentation

Prepared by:

Omar Ezz-ElDein Abdullah	20220228
Youssef Ahmed Awad	20220385
Ali Abdulrahman Mohamed	20220213
Youssef Tarek Ali	20220397
Mohamed Amr Mahmoud	20220302

Supervised by:

Dr. Eman Ahmed
Dr. Ali Zidane
Dr. Heba Sherif
Dr. Ghada Dahy

Academic Year: 2025/2026

Table of Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
CBAM	Convolutional Block Attention Module
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset format)
CPU	Central Processing Unit
CSV	Comma-Separated Values
FCAI	Faculty of Computers and Artificial Intelligence
GMP	Global Max Pooling
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
mAP	mean Average Precision
ML	Machine Learning
R-CNN	Region-based Convolutional Neural Network
REST	Representational State Transfer
RTL	Right-To-Left
UI	User Interface
YOLO	You Only Look Once

Abstract

English Abstract

Seed quality decides how well a crop germinates and how much it finally yields, yet it is still judged by hand. An inspector pours out a sample, spreads it on a tray, and sorts the seeds into good and bad piles one at a time. That work is slow, it tires the eye, and two people often disagree on the same seed, so it can only ever check a tiny part of a large batch. Seed Bank is a prototype that does this job automatically. A user takes a photo of a batch of seeds; the system finds every seed in the picture, decides whether each one is good or bad, and reports how many seeds there are, what share of them are good, and a breakdown by crop. It currently handles two crops, coffee and maize. Teaching a computer to find the seeds needs many example photos in which every seed is already marked, and those are expensive to prepare by hand. A companion tool, MultiSeedGen, solves this by building such training pictures automatically, so the system can learn without anyone marking thousands of photos. On unseen test images the prototype finds nearly every seed and grades each one correctly about 97% of the time, and it does so on an ordinary computer. This shows that fast, consistent, automatic seed grading is within reach of the small laboratories and farmers who cannot afford industrial sorting machines.

Arabic Abstract

تُحدّد جودة البذور إلى حدٍّ كبير نسبة الإنبات والمحصول النهائي، ومع ذلك ما زال تقييمها يجري يدويّاً. يسكب المُصنِّف عيّنةً من البذور، وينشرها على صينية، ويفرزها بذرةً بذرة إلى سليمة وتالفة. هذا العمل بطيء ويُجهد العين، وكثيراً ما يختلف شخصان على البذرة نفسها، ولذلك لا يمكنه فحص سوى جزء ضئيل من الدُّفعة الكبيرة. «Seed Bank» نموذج أوّليّ يؤدّي هذه المهمّة تلقائيّاً؛ إذ يلتقط المستخدم صورةً لدُفعة من البذور، فيعثر النظام على كلّ بذرة في الصورة، ويقرّر ما إذا كانت سليمة أو تالفة، ثمّ يعرض عدد البذور ونسبة السليم منها وتوزيعاً بحسب نوع المحصول. ويتعامل النظام حاليّاً مع محصولين، هما البُنّ والذُّرة. ويحتاج تعليمُ الحاسوب العثورَ على البذور إلى عدد كبير من الصور التي وُسِمت فيها كلّ بذرة مسبقاً، وهي صور مكلفة الإعداد يدويّاً. تسدّ الأداة المرافقة «MultiSeedGen» هذه الحاجة ببناء صور التدريب هذه تلقائيّاً، فيتعلّم النظام دون أن يَسِم أحدٌ آلاف الصور. وعلى صور اختبار لم يَرَها النظام من قبل، يعثر النموذج الأوّليّ

على كلّ البذور تقريباً، ويصنّف كلّ بذرة تصنيفاً صحيحاً نحو ٪97 من الحالات، ويفعل ذلك على حاسوب عاديّ. وهذا يبيّن أنّ تقييم جودة البذور تقييماً سريعاً ومتّسقاً وتلقائيّاً صار في متناول المختبرات الصغيرة والمزارعين الذين لا يقدرون على شراء آلات الفرز الصناعية.

Chapter 1

Introduction

1.1 Background and Motivation

Seed quality decides a large part of what a farmer harvests. A sample that looks fine to the eye can still carry a high fraction of broken, immature, discoloured, or diseased grains, and that fraction sets the germination rate and the final yield. Before a lot is planted, traded, or priced, someone has to judge how good it is. In most of the supply chain that judgement is still made by a human grader who pours out a sample, spreads it on a tray, and sorts the seeds by hand into good and bad piles, counting as they go.

Industrial optical sorting machines exist to automate that judgement, but they are not a general solution. They require a large capital outlay that puts them out of reach for smaller laboratories, independent researchers, and individual farmers. Once installed they demand continuous specialist maintenance to hold grading accuracy, and despite their scale they remain labour-intensive to operate and configure for new crop types.

Manual inspection has two problems that are hard to fix by training people better. It is slow: a careful grader works through a few hundred seeds before fatigue sets in, which is a tiny sample of a multi-tonne lot. It is also subjective. Two graders looking at the same tray will disagree on the borderline seeds, and the same grader will drift over a long shift. The result is a quality number that is neither repeatable nor auditable, which is a weak basis for a price or a planting decision.

There is also a technology gap. Large operations use industrial optical sorting machines, but those are expensive and complex enough to be out of reach for smaller laboratories, independent researchers, and individual farmers. On the other side of the gap there are no digital tools at all, so smaller operations fall back on subjective estimation or inconsistent sampling. Nothing affordable sits in the middle, and a large part of the agricultural community has no access to precision quality control.

Automating the task is harder than it sounds, because seeds are not manufactured parts on an assembly line. They are organic and irregular, they vary in texture and colour, and in a real

sample they touch and overlap. General-purpose detectors struggle in those dense clusters: two touching seeds get read as one, and subtle surface defects get missed. A grader that works on real trays has to handle that clutter rather than assume one clean seed per frame.

Computer vision still fits this task well once the clutter is taken seriously. The decision a grader makes, deciding whether one seed is good or bad, is a per-object visual classification, and the wider job of finding every seed in a tray is object detection. Both have matured to the point where they run on commodity hardware and beat a tired human on consistency. Surveys of deep learning in agriculture report strong results across crop and produce inspection once enough labelled data is available [1], and the same convolutional models that classify plant disease from leaf photographs transfer to grading the seeds themselves [2]. An automated grader does not get tired, scores every seed in the photographed sample, and produces the same answer twice for the same image.

This project, *Seed Bank*, builds that automated grader as a working prototype rather than a notebook demo. A user photographs a batch of seeds; the system locates each seed, scores its quality, and reports the aggregate good-rate, seed count, and a per-crop breakdown.

1.2 Problem Definition

Two linked problems sit at the centre of this project. The first is the shape of the machine-learning task itself. The second is a data problem that the first one exposes, and it is the harder of the two.

1.2.1 Morphological and Environmental Variability in Seeds

Evaluating seed quality is difficult partly because of the biological material itself. Seeds of the same species and quality class vary in shape, aspect ratio, surface area, and texture, and these differences interact with factors outside the seed: the angle at which it lies, the lighting at the moment the photograph is taken, and the visual character of the surface beneath it. Coat pigmentation shifts under different illumination, and natural shading or surface sheen can produce patterns that resemble defects without indicating any real damage. The result is that the visual boundary between a safe biological deviation and a true physical defect is often subtle and context-dependent. A classifier that cannot account for this overlap will either reject sound seeds as damaged or pass defective ones as acceptable, and neither error is acceptable in a quality-control setting.

1.2.2 The dataset gap

The first problem is volume. Seed quality assessment is a fine-grained visual task: the meaningful signal is often a hairline crack, a patch of discolouration, or a slight surface irregularity that separates a good seed from a damaged one. Models trained on fine-grained distinctions of

this kind require substantially more labelled examples per category than coarse classifiers do, with datasets in the range of one hundred thousand images per seed type providing a reasonable foundation for generalisation. The best openly available datasets for any given seed type fall well short of twenty thousand images, a gap wide enough that hand-collection alone is not a practical remedy.

The second problem is annotation bias. The public datasets that do exist were built for one purpose and annotated accordingly. Datasets assembled for quality grading carry per-image labels indicating whether a seed is acceptable or defective, but they provide no spatial information about seed location or extent. Datasets assembled for detection and counting annotate each seed with a bounding box, but they treat all seeds as a single undifferentiated class and record nothing about quality. No publicly available resource provides both spatial annotations and quality labels on the same images, which forces any work that needs both to either discard one annotation type or invest in a fresh labelling campaign.

The third problem is domain mismatch. Existing datasets were collected under controlled imaging conditions: uniform backgrounds, stable and diffuse lighting, and seeds placed individually or in sparse arrangements that minimise overlap. A model trained on such data learns the recording conditions alongside the seed appearance. When deployed against images taken in less controlled settings, with varying illumination, textured surfaces, and seeds that touch or partially occlude one another, accuracy falls because those conditions were absent from training. The controlled-environment bias reflects the practical difficulty of annotating cluttered scenes at scale, but the consequence is that published benchmark figures tend to overstate the accuracy a model will achieve in realistic use.

1.3 Project Objectives and Proposed Solution

The objectives follow directly from the two problems above:

- Build a complete grader that takes a photo or video of a seed batch and returns a grading report for seeds.
- Keep a record of which trained model produced each result, so any verdict can be traced back later.
- Deliver the grader as a web app that a farmer can actually use.
- Provide basic model management and a way to score a model against a labelled dataset before it is used.
- Close the data gap by generating the marked many-seed training photos the finding step needs.

The solution has two parts that match the two groups of objectives.

Seed Bank is the platform and prototype. A user uploads photos or videos through a web page; behind it the system finds the seeds, grades each one, stores the results together with a note of which trained model produced them, and sends back the report. A small amount of model management sits around this, enough to keep track of model versions and to check a model against a labelled dataset before it is put to use.

MultiSeedGen is the data generator. It cuts individual seeds out of source photos, drops many of them onto realistic backgrounds with varied lighting and a touch of camera-like noise, and saves the result as a ready-to-train dataset with every seed already marked. Pasting real cut-outs onto varied backgrounds is a known and effective way to produce labelled training images cheaply [3], and varying the lighting, scale, rotation, and noise follows the domain-randomisation idea of pushing a model to generalise by showing it a wide spread of synthetic variation [4]. Because the tool places every seed itself, it knows exactly where each one is, so the labels come for free.

1.4 Project Scope and Limitations

In scope. The grader covers multiple crops and is built so a new crop can be added by registering a new detector class and a matching classifier rather than by rewriting the pipeline. It is delivered as a working prototype with a web interface, and it supports offline evaluation of a trained model against a labelled dataset.

Limitations. Seed Bank is a prototype rather than a commercial product; the goal was to show the pipeline works end to end, not to ship a hardened deployment. Capture assumes a top-down photo or video from a single camera, so the system has not been validated against angled shots or varied capture rigs. The largest open risk is the synthetic-to-real domain gap: a detector trained only on MultiSeedGen output learns the statistics of synthetic scenes, which never match real trays perfectly. Narrowing that gap with augmentation and realistic degradation is the tool's central design concern, the limitation can be overcome by training the model on real data with real data which is time consuming and expensive.

1.5 Development Methodology

The work followed an iterative, phased approach. Rather than building everything at once, each phase delivered a usable slice and was exercised before the next was stacked on it: first the problem framing and the synthetic-data generator, then the grading pipeline, then the web prototype around it. The detector could not be trained until synthetic detection data existed, so MultiSeedGen came early and the platform work followed it.

The machine-learning side ran as a tighter experiment loop. The first step was curating and cleaning the datasets, since raw public corpora carry mislabelled and duplicated images that would otherwise leak between splits. From there the team trained a sequence of model variants and measured each one on validation data and on a held-out test set before deciding what to change next.

1.6 Tools and Technologies Used (Software and Hardware)

The stack is conventional where it can be, so that each choice is a well-understood tool with a known failure mode. The backend is built on Python and FastAPI [5]; the models are built and trained with PyTorch and torchvision, with Ultralytics YOLO and OpenCV alongside; relational data is stored in PostgreSQL, while analytics and telemetry data are managed in ClickHouse; the web client is React [6]; and the whole system runs under Docker [7]. Table 1.1 groups the set by area.

Table 1.1: Tools and technologies used, grouped by area.

Area	Tools
Backend	Python, FastAPI.
ML	PyTorch, torchvision, Ultralytics YOLO, OpenCV.
Data	PostgreSQL, ClickHouse.
Frontend	React.
Tooling	Docker.

On the hardware side, the models were trained using cloud based gpus and CPUs.

1.7 Project Timeline / Gantt Chart

The work was scheduled across the 2025/2026 academic year and ran in the phased order described in the methodology section. The early part of the year went to problem framing and to MultiSeedGen, because the detector could not be trained until synthetic detection data existed. The middle of the year built the grading pipeline and ran the model experiments, and the final stretch went to the web prototype and this report. Figure 1.1 shows the schedule.

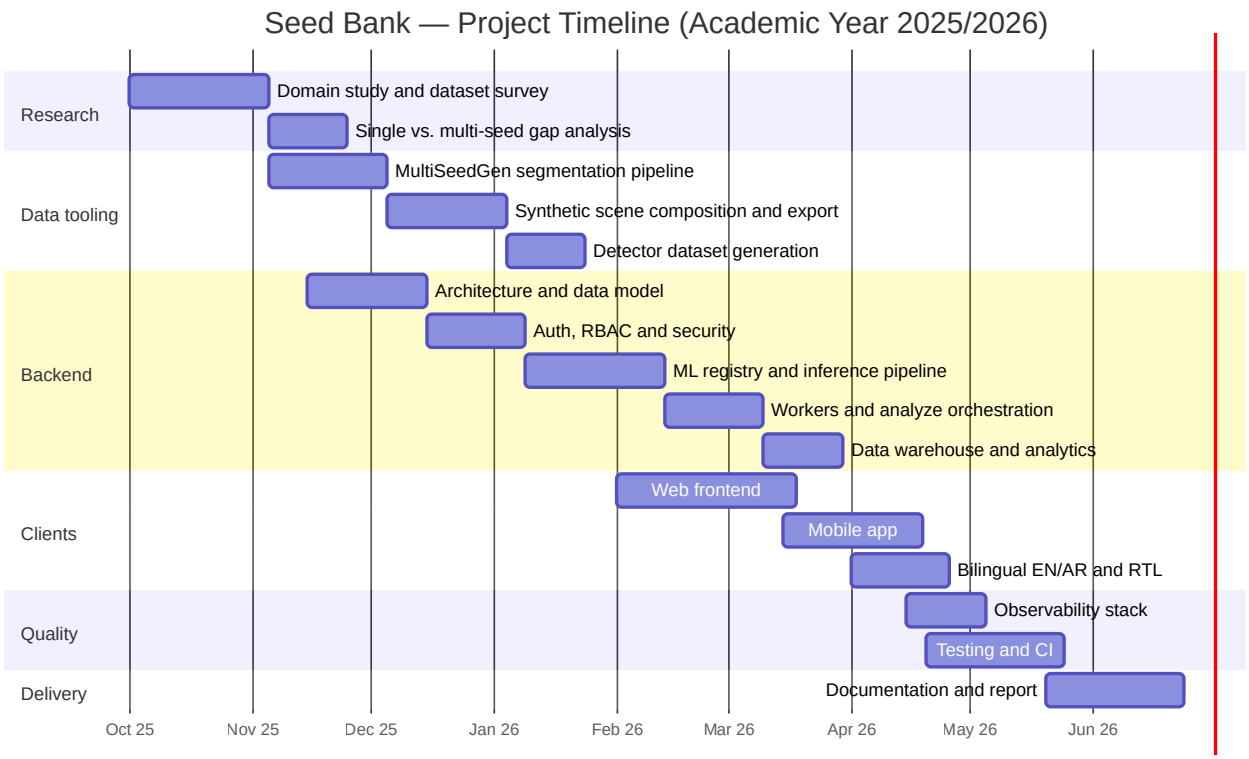

Figure 1.1: Project timeline (Gantt chart).

1.8 Report Organization

The rest of this report is organised as follows.

Chapter 2, Related Work, reviews the background this project builds on: object detectors such as Faster R-CNN and YOLO, image classifiers, the synthetic-data and augmentation literature that justifies MultiSeedGen, and prior applications of computer vision in agriculture.

Chapter 3, System Analysis, sets out the requirements. It defines the user roles, the functional and non-functional requirements, and the use cases the grader has to support, and frames them against the two problems from this chapter.

Chapter 4, System Design, describes the architecture: the grading pipeline, the data model, and the web client.

Chapter 5, Implementation, covers how the design was actually built, the model builders and inference path, the orchestration of the detect-then-classify flow, and MultiSeedGen's generation pipeline.

Chapter 6, Experimental Results and Evaluation, reports the experiments: the datasets and how they were prepared, the detection and classification metrics across the model variants, and the latency of the end-to-end pipeline.

Chapter 7, Conclusion and Future Work, summarises what was delivered against the objectives and lays out the next steps, prioritising the synthetic-to-real domain gap.

Chapter 2

Related Work

This chapter sets out the background the project builds on. It starts with how seed quality is checked today, by hand, in a working seed bank. It then surveys five existing systems that automate part of that job, and notes where each one falls short of what we need. From there it moves to the computer-vision ideas behind our pipeline (object detectors and attention classifiers), the synthetic-data techniques that let us train a detector without labelling thousands of seeds by hand, and a short review of where AI sits in agriculture more broadly. The chapter closes with a comparison of prior work against what Seed Bank delivers.

2.1 Traditional methods of seed analysis

Traditional seed grading relies on mechanical systems that filter seed streams through grids, sieves, and gravity tables, separating cohorts by size, weight, and density into discrete quality tiers. These installations categorise large volumes of seed into grades, typically spanning premium, first, second, and lower-tier classifications, without requiring a human to examine each kernel.

These systems grade by physical proxies size, weight, and ensity so a cracked or discoloured kernel that falls within the acceptable mass range passes through undetected. Beyond that accuracy ceiling, the installations carry steep drawbacks: the capital outlay for industrial sorting equipment runs to tens of thousands of dollars, placing it out of reach for smaller seed banks, independent laboratories, and individual farming operations. Ongoing specialist maintenance is required to hold calibration, reconfiguring for a new crop or a revised grading standard demands physical intervention, and the throughput advantage is partly offset by the labour needed to operate and oversee the lines. The result is a technology that serves large commercial operations but is inaccessible to most of the agricultural sector.

2.2 Existing solutions

To understand the current landscape we looked at five systems, ranging from industrial machinery to open-source research code. Each automates some part of seed inspection, but most of them target growth potential (germination) or species identification rather than the physical quality checks that long-term storage actually requires.

2.2.1 LemnaTec SeedAIxpert

The LemnaTec SeedAIxpert [8] is the reference point for large corporate seed labs and plant breeders. It is an all-in-one hardware and software product built around an imaging cabinet with controlled lighting, so that every photograph is taken under identical conditions and the AI software can produce reproducible scores. It is designed for high throughput, processing large quantities of seed and replacing subjective human grading with standardised numbers.

Relevance to our project: the strength of the system is also its barrier, which is accessibility. It depends on custom, costly hardware and careful calibration that is impractical for a typical university seed bank or a smaller conservation effort. Its closed design also means we cannot adapt its algorithms to the specific physical defects we care about. Our aim is to reach useful accuracy from ordinary photographs without a dedicated imaging rig.

2.2.2 PCS Agri Track

PCS Agri Track [9] is a commercial product aimed at nurseries, built to modernise inventory management and germination tracking. Unlike the industrial lab equipment it runs on mobile and web, which lowers the barrier to entry. It moves nurseries away from manual counting by digitising the tracking process and recording data over time, such as germination curves.

Relevance to our project: the main drawback is its reliance on cloud processing and a stable internet connection, which is not always available in the field. Users have also reported that the interface is hard to follow and that the models generalise poorly, working for common nursery seeds but struggling with the diverse or rare species a seed bank holds. It still needs manual intervention for edge cases, which undercuts the goal of automation.

2.2.3 Seedy

Seedy [10] is a mobile app for hobbyist gardeners, botanists, and educators. Its purpose is on-demand seed identification and personal collection management. A user photographs a seed with a phone camera, and the app's recognition model returns the species along with metadata and growing guides.

Relevance to our project: Seedy shows that mobile seed analysis is both viable and easy to use, which supports our own mobile capture flow. Its scope, though, is strictly identification. It

tells the user what a seed is, not how healthy it is. It does not measure physical metrics or scan for damage and disease, which is exactly the gap we are trying to fill.

2.2.4 GerminationPrediction

In the academic sphere the GerminationPrediction project of Grimm et al. [11] is a useful reference. It is open source, uses a Faster R-CNN detector trained on a public dataset of over twenty-three thousand images, and automates germination assessment by counting how many seeds in a batch have sprouted.

Relevance to our project: this is the closest match technically, since it also uses a CNN-based detector. The biological focus is different, though. They measure viability (whether a seed will grow), while we measure physical integrity (whether a seed is sound enough to store). A seed can be viable enough to sprout and still carry cracks or fungal spores that make it risky to store in a shared freezer. We borrow their open-source, reproducible approach but retrain the focus toward defect detection rather than sprout counting.

2.2.5 Multi-View Oil Palm system

The Multi-View Oil Palm project [12] pushes computer vision past the limits of a single image. It uses Multi-View CNNs (MVCNN) to analyse irregularly shaped oil palm seeds from several angles, which gives it a 3D understanding and strong accuracy on that crop.

Relevance to our project: this work makes a point we take seriously, that a single top-down photograph can miss defects hidden on the side or underside of a seed. Their multi-view setup gives richer data, but it is specialised for oil palm and needs a complex rig to capture multiple angles. We aim for a middle ground: high accuracy from accessible single-view photographs, without the computational and hardware cost of a multi-view system.

2.3 AI in agriculture

AI is now a routine part of modern agriculture rather than an experiment. The shift is driven by pressure on the sector: any land is shrinking and climate conditions are more volatile, so the older ”yield at all costs” mindset has given way to a precision-first one, where efficiency and resilience matter as much as raw output. The survey by Kamilaris and Prenafeta-Boldú [1] documents this move across agriculture, from yield prediction to weed and crop classification to quality grading, all shifting from hand-crafted features to learned representations with steady accuracy gains where labelled data was available.

In precision farming, models combine satellite imagery, soil-sensor readings, and local weather forecasts into field-level prescriptions for how much water or input each area needs. Closer to our problem is fine-grained computer vision for quality control. Where ordinary

detection just identifies a crop type, fine-grained methods pick up subtle within-class differences such as hairline fractures, surface discoloration, or early-stage fungal damage that often escape the human eye. Mohanty et al. [2] show both the promise and the catch: their CNNs reached high accuracy at recognising crop diseases from leaf photographs on a curated dataset, but accuracy dropped sharply on images taken under conditions different from the training set.

That last result points to the barriers the field still faces. The first is capital cost: sensors and autonomous machinery carry a high upfront price that keeps smaller operations out. The second is the digital divide, a widening gap between fully integrated commercial farms and smallholders. The third is the one that shaped our own work most: dataset variability and domain generalization. Seed diversity and lighting changes break models trained on a narrow set, and the usual fix is to collect local data and retrain for the specific harvest. The gap between a clean benchmark and field photographs is the central practical problem for any agricultural vision system, and it is the reason we treat data variety as a first-class concern rather than an afterthought.

2.4 Comparative analysis

The components above are individually well established. What distinguishes this project is not a new model but how the pieces are assembled into one workable pipeline, and the decision to remove the data-labelling bottleneck with a purpose-built generator rather than working around it.

Most prior work in seed and grain vision is a single artifact: one classifier or one detector, trained against a hand-labelled dataset and evaluated once. Such a system answers one question, either ”where are the seeds” or ”is this seed good”, but not the combined question the domain actually asks, which is how many seeds are in this photo and how many of them are bad. It is usually trained on a hand-labelled dataset, which caps the dataset size and therefore the accuracy.

Seed Bank differs on each point. It chains a detector and a per-type classifier into one detect-then-classify pipeline, so a single photograph yields a located, per-seed good/bad verdict and an aggregate report. It supports multiple crop types, each with its own classifier, with detections grouped by type before the classification stage. And the labelling bottleneck is attacked at the source by MultiSeedGen, which generates perfectly labelled multi-seed data with controlled variety, so the detector can be trained at a scale that hand-labelling could not reach. Table 2.1 summarises the contrast.

Table 2.1: Typical prior work in seed and grain vision versus the Seed Bank platform.

Dimension	**Typical prior work**	**Seed Bank**
Scope	Single model: a detector *or* a classifier	Two-stage detect-then-classify pipeline producing per-seed verdicts and an aggregate report
Quality question answered	"where are the seeds" *or* "is this seed good"	"how many seeds, and how many are bad" over a mixed photo
Crop types	Usually one species, fixed at training	Multiple crop types, each with its own classifier; detections grouped by type
Training data	Hand-labelled, which caps dataset size	Synthetic cut-and-paste generation with free, exact labels and controlled variety mixed with real data
Data integrity	Leakage and label errors are easy to miss	Leakage-safe split at the source-seed level, with controlled overlap and box clipping

The wider point is that the contribution sits at the system level. Each model family here is taken from published work and used as intended. The originality is in binding detection, per-type classification, and a synthetic-data generator into one tool that a non-researcher can run, and in attacking the labelling bottleneck head-on rather than living with a small hand-labelled dataset. The chapters that follow describe how that system is built and how it performs; the experimental numbers are reported in the evaluation in Chapter 6.

Chapter 3

System Analysis

This chapter turns the product idea from the introduction into a clear statement of what Seed Bank has to do and the limits it works within. It starts with the people who use the system and what each of them wants, then lists the functional and non-functional requirements that the design is measured against. The requirements map onto capabilities that exist in the running system, so the chapter also acts as a bridge between what the stakeholders need and what the backend, the workers, and the user interface actually provide.

3.1 Stakeholder Analysis

Seed Bank serves two groups that have little in common. On one side are the people checking seed quality in a field or a warehouse. On the other are the people who build and look after the machine-learning model behind the product. Both groups depend on the same data, because the model that grades a farmer's photo is the same one a developer trained, registered, and evaluated. Naming each stakeholder and their goals early kept the requirements grounded, since a feature that helps one group should not break what the other relies on.

The farmer, or end user, is the main stakeholder. Their goal is to find out whether a batch of seeds is good before planting or buying it. They sign in from the web app or the mobile app, photograph a batch, and expect a quick good/bad report with a seed count and a good-rate, plus a history they can return to. Farmers often work on a phone and in Arabic, so the interface language and text direction matter as much as the result itself.

Seed quality assurance companies form a second category of institutional stakeholder. These organisations currently run mechanical sorting lines sieves, gravity tables, and weight-based separators that are expensive to procure, require continuous specialist calibration, and cannot detect surface defects on seeds that fall within the acceptable mass range. Seed Bank offers these operators a different approach: a conveyor-mounted high-resolution capture system feeds images into the pipeline, which grades each seed visually in real time and logs the results. The mechanical infrastructure is replaced by a camera rig and the hosted service, maintenance is

reduced to software updates, and the grading record is stored and auditable rather than lost on the sorting floor.

The AI developer keeps the product accurate over time. They train the detection and classification models, register the resulting weights, and run offline evaluations against labelled datasets to decide whether a new model is actually better than the one in use. Because they need to test a specific model on a real photo, they can pick which model runs at analysis time, which an ordinary user cannot do.

The administrator looks after the operational side. They choose which model serves traffic, manage users and their roles, and review the record of sensitive actions. An administrator can do everything an AI developer can, plus the settings that change behaviour for everyone, so those actions are gated tightly and logged.

Table 3.1 summarises each stakeholder, their main goal, and the capabilities they lean on.

Table 3.1: Stakeholders, their goals, and the capabilities they depend on.

Stakeholder	Primary goal	Depends on
Farmer / end user	Check the quality of a seed batch in the field and review past results	Web/mobile capture, batch analysis, history, English/Arabic UI
AI developer	Train and validate models, then choose the right one	Model registry, datasets, offline evaluation, per-request model choice
Administrator	Control which model serves traffic and manage access	Model selection, user/role management, action log
QA company	Replace mechanical sorting lines with a conveyor vision system and retain an auditable grading record	Batch analysis, real-time throughput, grading history, model registry

3.2 Functional Requirements

The functional requirements describe what the system does, grouped by the part of the product they belong to. Each one corresponds to a capability in the running service, so they can be traced into the design and checked against it.

3.2.1 Accounts and access

A user can register with an email and password, confirm their address, and sign in. They can also sign in through Google. Once signed in, a user can manage their own account. Access is split across three roles (end user, AI developer, administrator), and each requirement below records the role allowed to perform it.

3.2.2 Analysis and history

The core action is submitting images for analysis. A user uploads a batch of images, optionally noting the seed type and supplier, and the system returns a good/bad verdict per seed along with a count and a good-rate for the batch. A faster mode trades some detail for a quicker turnaround on large batches. A user can also analyse a single image on its own. Around a batch, a user can list their batches, open one to see the aggregated scans, detections, statistics, and annotated images, export the results, download an image with the detection boxes drawn on it, and delete batches they no longer need.

3.2.3 Model work

AI developers and administrators look after the model. They register trained model weights along with their metadata, list and fetch them, and read how each model performed. They manage labelled datasets and run offline evaluations that score a model over a dataset. An AI developer can also choose which model runs for a given analysis, so a candidate model can be tried on a real photo before it takes over.

3.2.4 Administration

Administrators choose which model serves normal traffic, manage the user base by listing users and changing their roles, and read the log of sensitive actions.

Table 3.2 gives each functional requirement an identifier and the role permitted to perform it.

Table 3.2: Functional requirements with the role permitted to perform each.

ID	Requirement	Role
FR-1	Register with email/password and confirm the address	Public
FR-2	Log in and stay signed in across sessions	Public
FR-3	Sign in with Google	Public
FR-4	Submit a batch of images for analysis	End user
FR-5	Use the faster analysis mode for large batches	End user
FR-6	Analyse a single image on its own	End user
FR-7	List own batches and open one for detail	End user (own)
FR-8	View batch detail: scans, detections, statistics, and images	End user (own)
FR-9	Export a batch's results	End user (own)
FR-10	Download an annotated image with detection boxes	End user (own)
FR-11	Delete a batch	End user (own)
FR-12	Check service health and read runtime configuration	Public

ID	Requirement	Role
FR-13	Register a model's weights and metadata; list and fetch models	AI developer
FR-14	Read per-model performance	AI developer
FR-15	Manage labelled datasets and run offline evaluations	AI developer
FR-16	Choose which model runs for a given analysis	AI developer
FR-17	Choose which model serves normal traffic	Administrator
FR-18	List users and change a user's role	Administrator
FR-19	Read the log of sensitive actions	Administrator

3.3 Non-Functional Requirements

Non-functional requirements describe how the system should behave while doing the work above. They cover the qualities a user or reviewer would notice if they were missing.

Performance. Analysis does not block the request. When a user submits a batch, the system accepts the images, starts the work in the background, and lets the client check back for the result. The images in a batch are processed independently rather than one after another, so a multi-image batch finishes faster than the sum of its parts. The faster analysis mode exists for cases where turnaround matters more than the extra detail.

Usability. The end-user surface is simple enough to use on a phone in the field. A user photographs a batch, waits a moment, and reads a clear good/bad result. History, exports, and annotated images are reachable in a few taps.

Internationalisation and RTL. The end-user interface works in both English and Arabic, including right-to-left layout. The layout flips correctly so dialogs, tables, and navigation read naturally in either language. The developer and admin pages are English-only, since farmers never reach them.

Security. Sign-in keeps users in their own data, and the three roles decide what each user can do, enforced on the server rather than just hidden in the interface. Sensitive actions are written to a log that can be reviewed but not edited after the fact.

Reliability. A batch moves through a clear set of states as it is processed. Several workers can handle the images of one batch at the same time without corrupting its state. If detection succeeds but classification later fails, the batch keeps the detections it already has instead of throwing them away, and a batch that cannot be matched to a model fails with a readable message.

Maintainability. The backend is layered so that each part has one job: request handling, business logic, and data access stay separate. Inference runs in a separate worker service rather than inside the web process, so the two can be changed and scaled on their own. Swapping the model that serves traffic is a registry-and-selection step, not a code change, which keeps the system easy to update as new crops and models are added.

Table 3.3 lists the non-functional requirements.

Table 3.3: Non-functional requirements.

ID	Attribute	Requirement
NFR-1	Performance	Analysis runs in the background; the images in a batch are processed in parallel.
NFR-2	Performance	A faster analysis mode is available for large batches.
NFR-3	Usability	The result is readable at a glance on a phone in the field.
NFR-4	Internationalisation	Full English/Arabic interface with right-to-left layout.
NFR-5	Security	Three roles enforced on the server; sensitive actions logged.
NFR-6	Reliability	Clear batch state flow; concurrent workers cannot corrupt a batch; partial results are kept on classification failure.
NFR-7	Maintainability	Layered backend; inference in a separate worker; model swap is a selection step, not a code change.

3.4 Use Case Diagram and Actor Descriptions

The use cases group into three areas, following the same split used in the first-semester design: system utilities, history and statistics, and seed analysis. The diagrams are split logically by actor: Figure 3.1 shows the Farmer/End User use cases, Figure 3.2 shows the AI Developer use cases, and Figure 3.3 shows the Administrator use cases.

System utilities cover the supporting calls that keep the service usable: a health check that reports whether the service is up, and a configuration call that lets a client read the current runtime settings.

History and statistics centre on viewing a batch in detail. Opening a batch pulls together its scan history, the individual seed detections, the aggregated statistics for the batch, and the annotated images into one view, so a user can see both the summary and the per-seed evidence behind it.

Seed analysis is the core of the system. A user analyses a batch of images in the standard mode, or uses the faster mode when a large batch needs a quicker turnaround. Both modes also work on a single image, which is useful for checking one sample or for tracing a result down to a single photo.

Three actors drive these use cases. The end user, a farmer or a person checking seed quality, runs analyses and reviews their own history. The AI developer does everything the end user can, and also trains models, registers them, runs offline evaluations, and chooses which model runs for a given analysis so a candidate can be tried on a real photo. The administrator does everything the developer can, and also chooses which model serves normal traffic, manages users and their

roles, and reads the log of sensitive actions. These last actions change behaviour for everyone else, so they sit with the most-privileged actor.

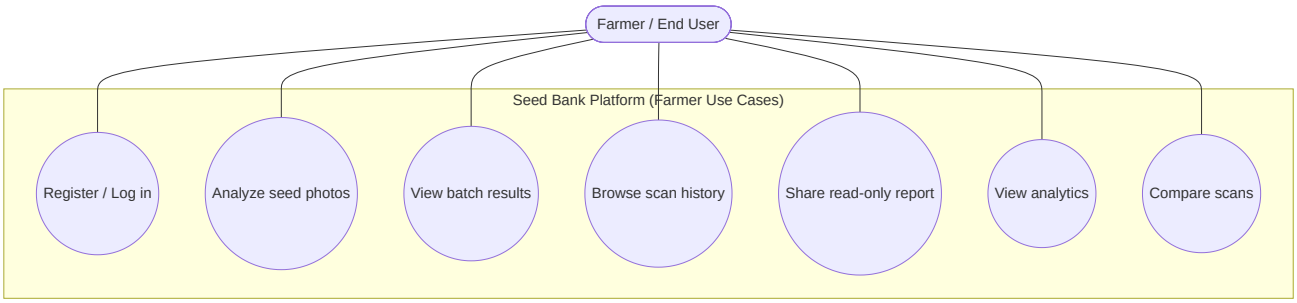

Figure 3.1: Use-case diagram: Farmer / End User.

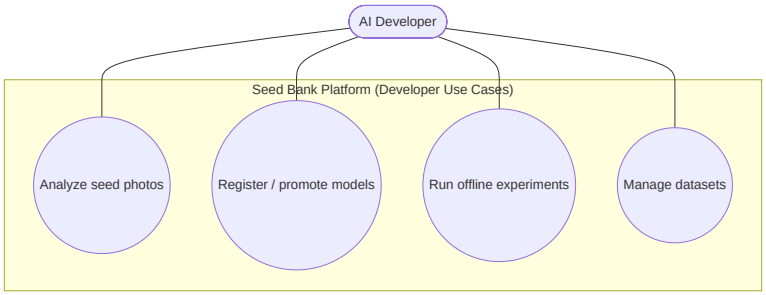

Figure 3.2: Use-case diagram: AI Developer.

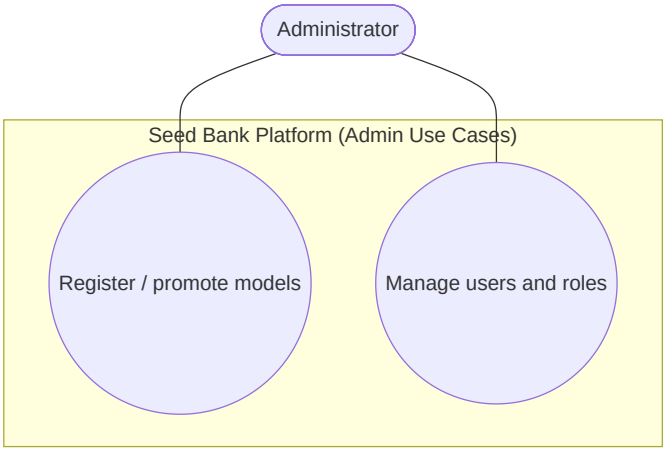

Figure 3.3: Use-case diagram: Administrator.

3.5 Feasibility Study

Technical feasibility. The whole stack runs on commodity hardware. The backend, the inference worker, the database, the object store, and the supporting services come up together on a single developer machine, with demo data seeded in. the demo, and offline evaluation all run on CPU,and a GPU mainly helps when training new models. Every major component is a mature open-source project with good documentation, so the build does not rest on any unproven

technology.

Economic feasibility. The cost case rests on open-source software and ordinary hardware. There are no licensing fees for any part of the stack, and the development and demo setup runs on one machine. A GPU is needed only for training or for heavy inference at scale. The object store and the model-tracking service are self-hosted alongside the rest, so there is no recurring spend on managed cloud services to run or evaluate the system.

Operational feasibility. The product meets each group where they already are. Farmers reach it through a web app and a mobile app that uses the device camera, both in English or Arabic with right-to-left layout, so the interface does not assume English literacy or a desktop. Developers and administrators reach the same backend through a documented API and the role-gated web pages. Both clients talk to one shared API, so the system behaves the same across web, mobile, and direct API use, which keeps it practical to run and to hand over.

Chapter 4

System Design

This chapter describes how the Seed Bank prototype is put together: the overall architecture, the object model behind the inference pipeline, the main request flow, the relational schema, the screens the user sees, and the security posture. It also covers MultiSeedGen, the companion tool that generates the synthetic training data the detector learns from. The design keeps the parts that change for different reasons apart, so a new model, a new table, or a new screen can be added without disturbing the rest [13].

4.1 System Architecture

At a high level the prototype has five pieces. A React web client is the front end the user works with. A FastAPI backend exposes the API and runs the two-stage inference pipeline that turns a seed photo into a report [5]. A relational database (PostgreSQL) holds users, scan batches, and the per-seed results. An analytical database (ClickHouse) stores the inference metrics, scan statistics, and system telemetry for performance tracking. File storage keeps the uploaded images themselves, so the databases store references rather than image bytes.

Model inference is the heavy part of the work, so it does not run inside the request that the web client is waiting on. The API accepts a batch, records it, and hands the actual detection and classification off to a separate worker service. The worker pulls the image, runs the pipeline, and writes the results back to the database. This keeps the API responsive: a user submitting a batch gets an immediate acknowledgement instead of waiting on a slow forward pass, and several images can be graded in parallel by the worker.

Figure 4.1 places the system in its setting, showing the actors and the boundary they talk to.

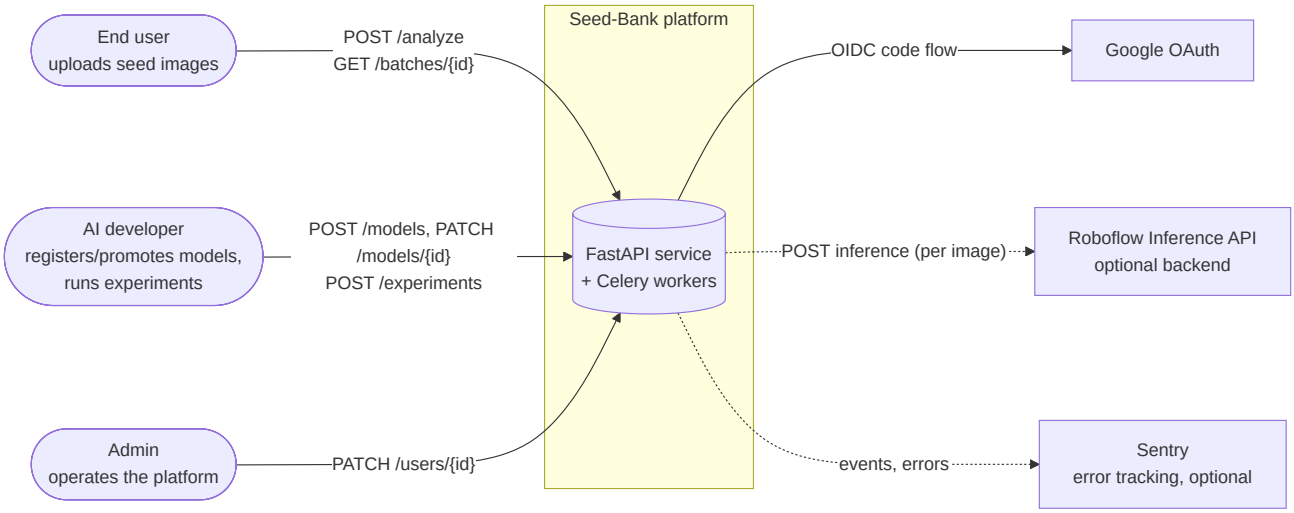

Figure 4.1: System context: the actors and the Seed Bank boundary they interact with.

The worker side handles the actual inference. Figure 4.2 shows the task orchestration and worker process loop. The detailed pipeline itself is split in two: Figure 4.3 shows detection through the persisted detect rows; Figure 4.4 shows classification through batch finalization.

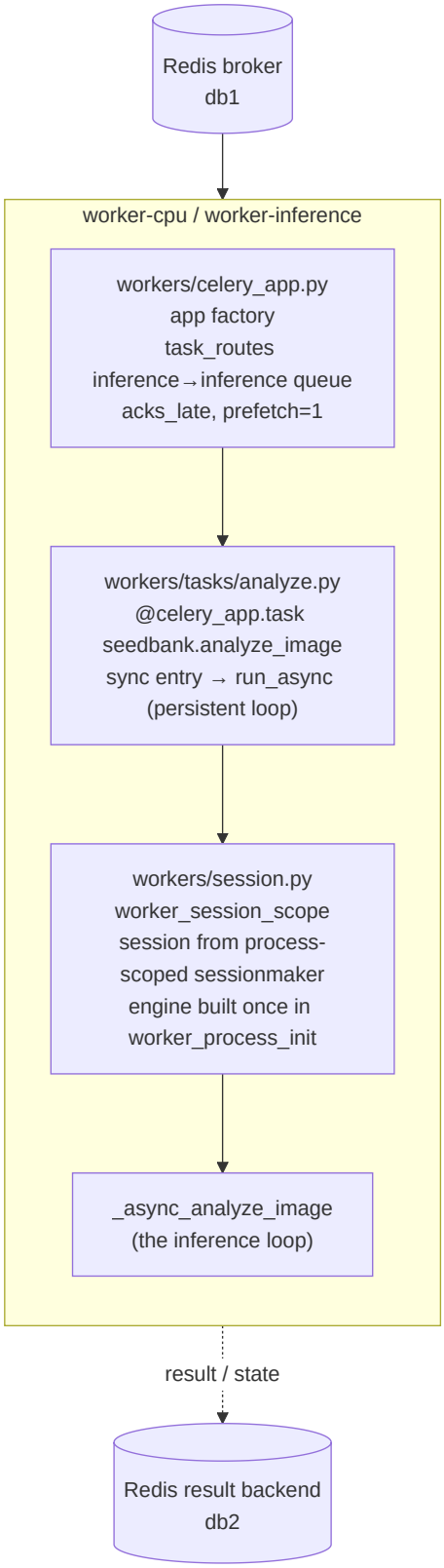

Figure 4.2: Inference worker task orchestration and dispatch.

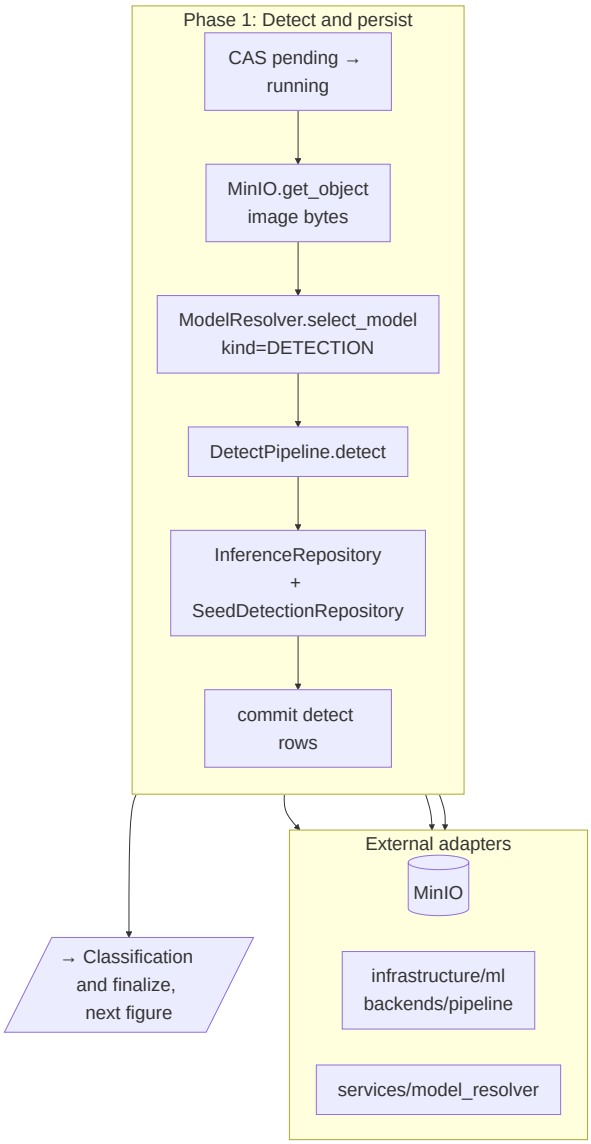

Figure 4.3: Inference worker pipeline: detection, from CAS pending → running through committing the detect rows.

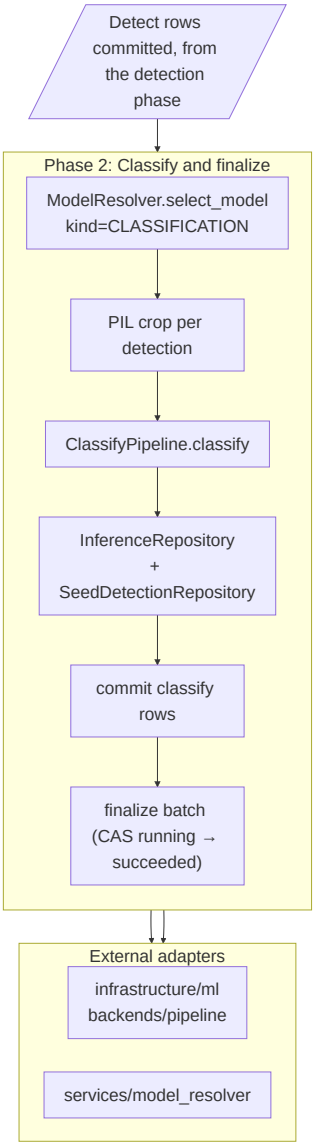

Figure 4.4: Inference worker pipeline: classification, from cropping each detection through finalizing the batch.

4.2 Class Diagrams

The object model behind the inference pipeline is split into two views so each stays readable at print size. Figure 4.5 shows the ML platform machinery: the `InferenceBackend` protocol implemented by the three interchangeable engines (local PyTorch, Ultralytics YOLO, Roboflow), the `ModelManager` that loads and LRU-caches weights, the `ModelBuilder` registry that constructs an architecture before weights are loaded into it, and the `ModelResolver` that decides which model version actually runs a request.

Figure 4.6 shows the request-facing side: `AnalysisService` is the entry point that dispatches a scan and persists its state through `ScanBatchRepository`, whose compare-and-set `cas_status` keeps concurrent workers from corrupting a batch's status. A backend (shown in

Figure 4.5) returns `Detection` and `Classification` objects for each request; a `Detection` carries a `BoundingBox`.

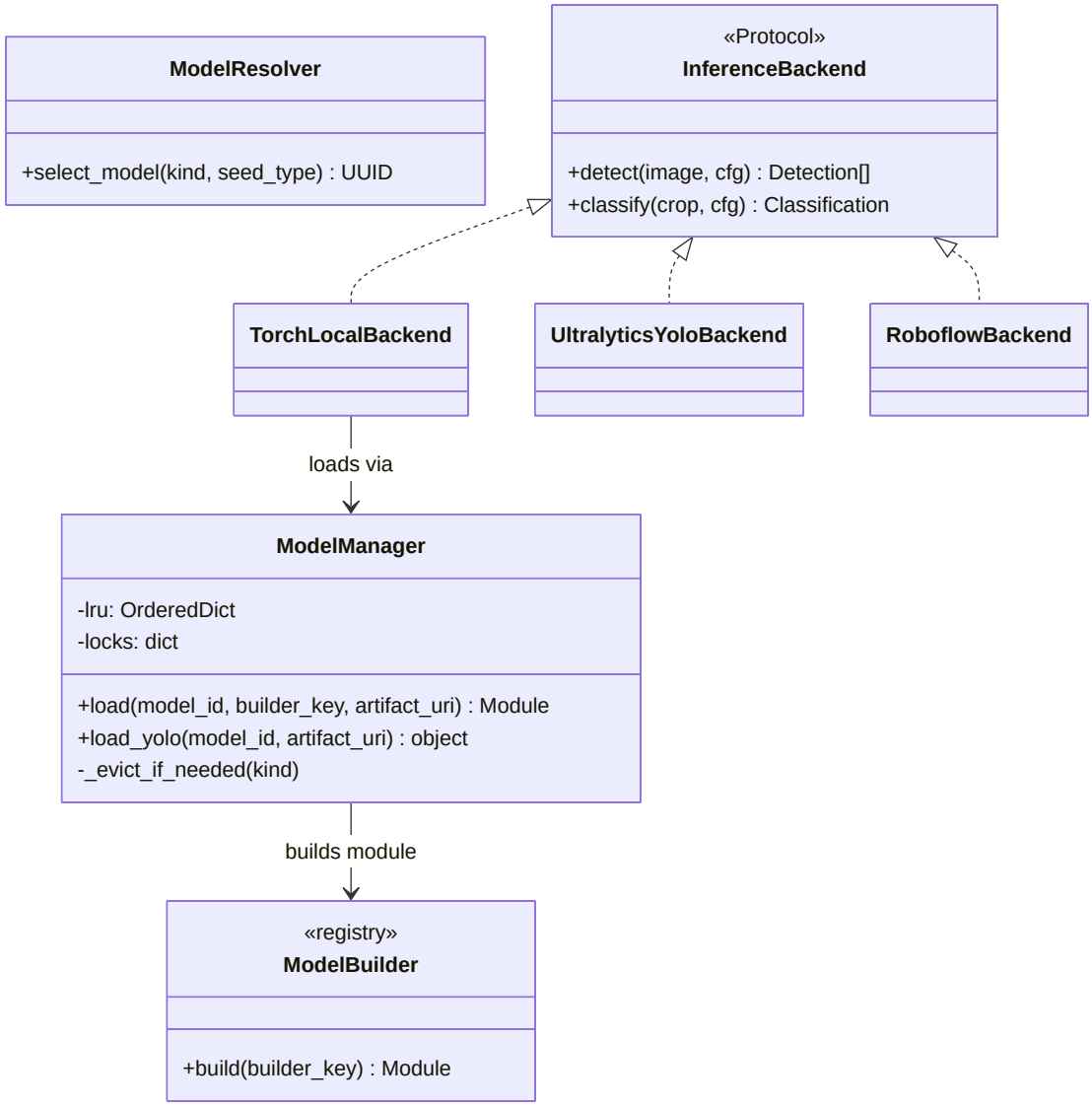

Figure 4.5: Class diagram: the `InferenceBackend` protocol and its three engines, backed by `ModelManager` for weight loading/caching and `ModelBuilder` for architecture construction, with `ModelResolver` selecting the production model.

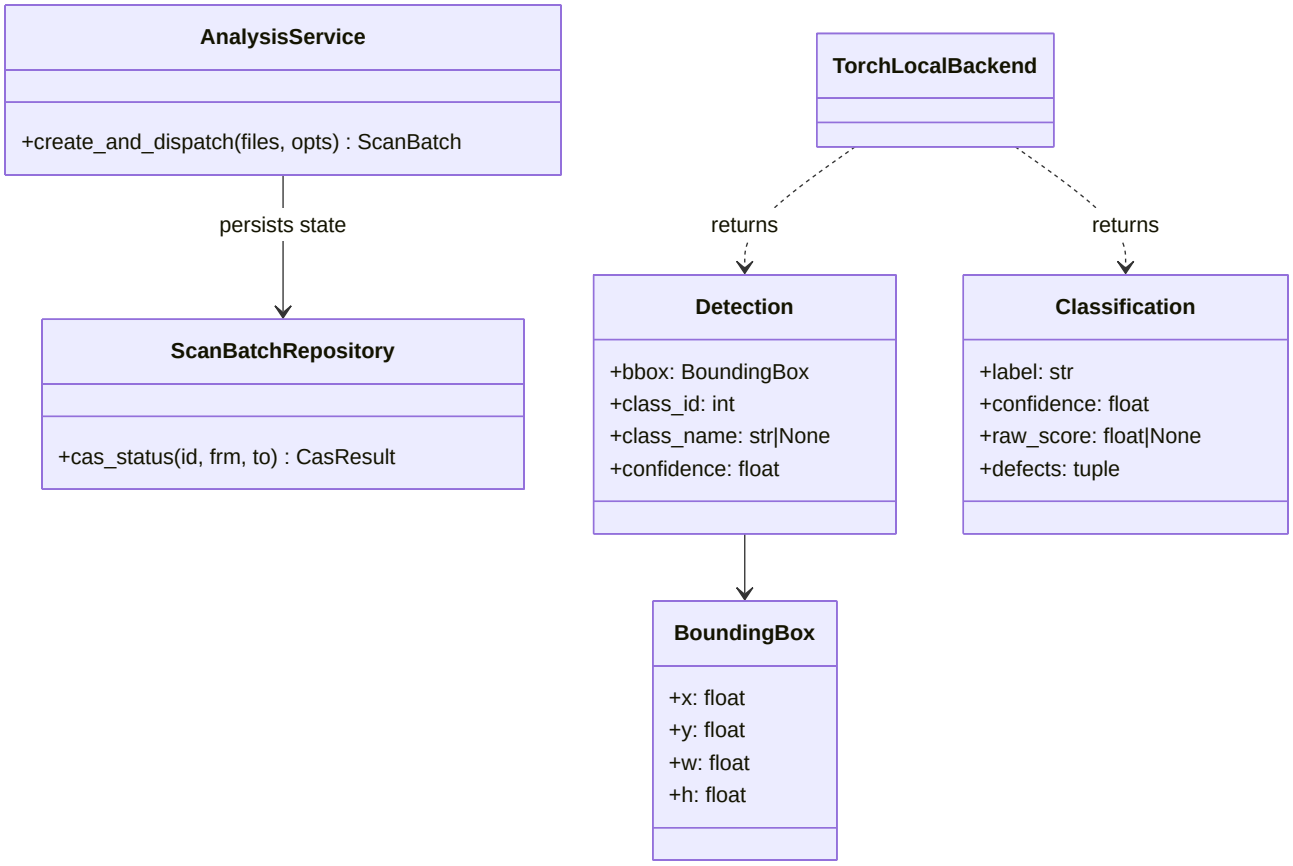

Figure 4.6: Class diagram: `AnalysisService` dispatching a scan through `ScanBatchRepository`, and the `Detection`/`Classification`/`BoundingBox` result objects a backend returns.

4.3 Sequence Diagrams

The main interaction is batch analysis, shown in Figure 4.7. When a user submits a batch, the backend first identifies the user, then creates a new `ScanBatch` record in the `PROCESSING` state. It iterates through the uploaded files, saving each one to file storage and creating a matching `ScanImage` entry.

Each image then goes through the pipeline. The detector locates every seed in the image, and the backend records one `SeedDetection` per seed with its bounding box. The system then crops each detected seed from the source image and classifies it as good or bad. When every image is done it aggregates the statistics across the batch, updates the status to `COMPLETED`, and returns the structured report to the client.

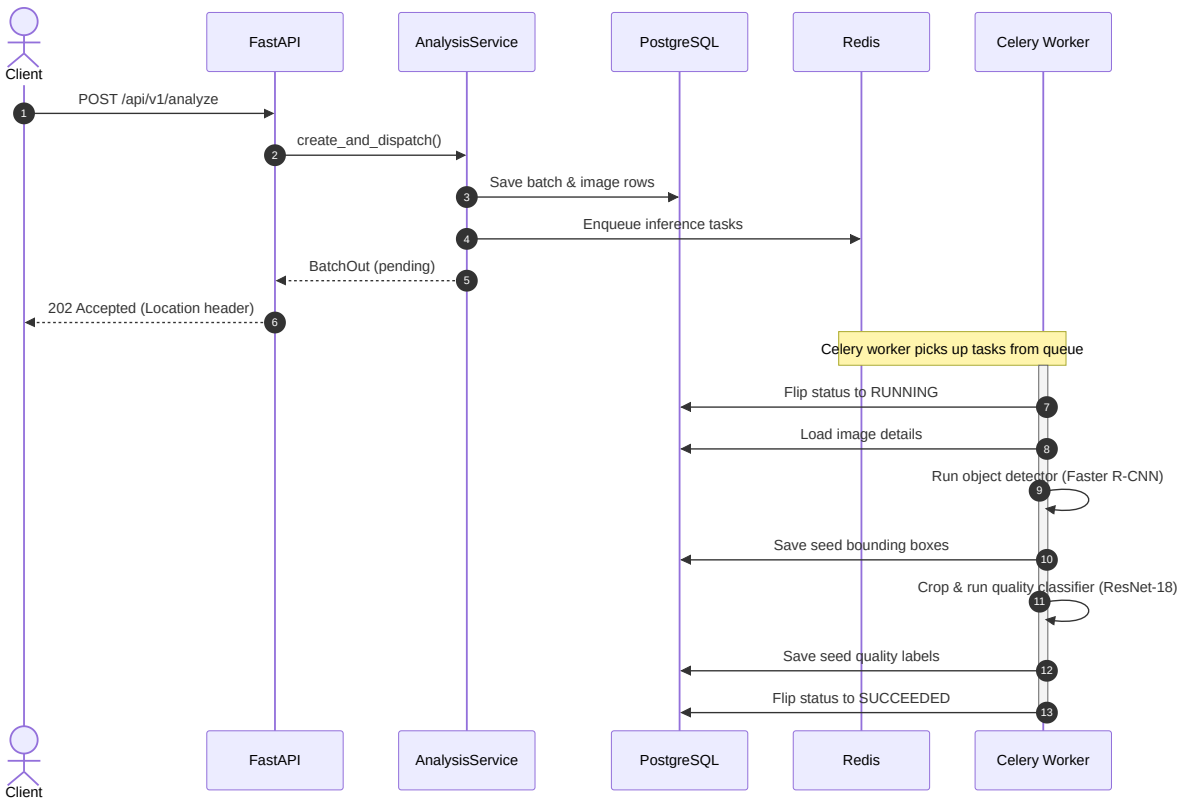

Figure 4.7: Batch-analysis sequence: identify the user, create the batch, save images, detect seeds, crop and classify each one, aggregate, and complete.

4.4 Database Entity-Relationship Diagram

The database schema is split into six focused views to preserve printed legibility. Figure 4.8 shows how a user authenticates (password and Google OAuth) and how a refresh token is issued; Figure 4.9 shows the append-only audit log and the suppliers a user creates. Figure 4.10 shows how a supplier's batch and its uploaded images are created; Figure 4.11 shows what one image's inference produces, down to the per-seed detection. Figure 4.12 shows how a model version is registered and scored; Figure 4.13 shows how a model is evaluated offline against a labelled dataset.

The core scan hierarchy spans four central entities:

- `USER` manages identity and access, and owns the scans a person creates.
- `SCAN_BATCH` is the primary operational container. It tracks the progress of an analysis request (pending, processing, completed, or failed) and aggregates summary metrics (total seed count, bad-seed count, average confidence).
- `SCAN_IMAGE` records the metadata for every individual file in a batch, including its storage path and its pixel dimensions.
- `SEED_DETECTION` is the granular output of the models, storing a quality label (good or bad) and the normalized bounding box coordinates.

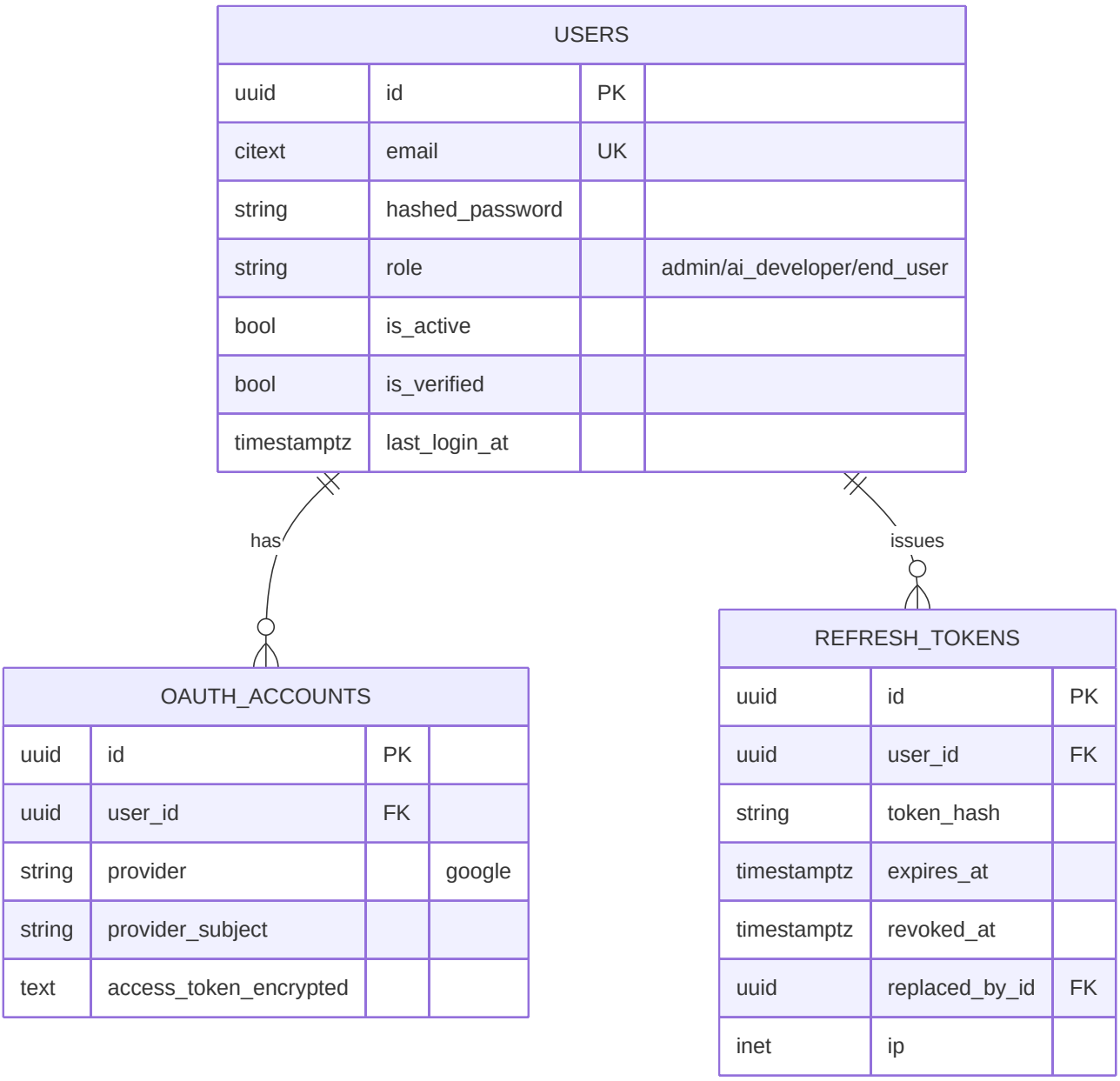

Figure 4.8: Database schema: user identity — password/OAuth login and refresh-token issuance.

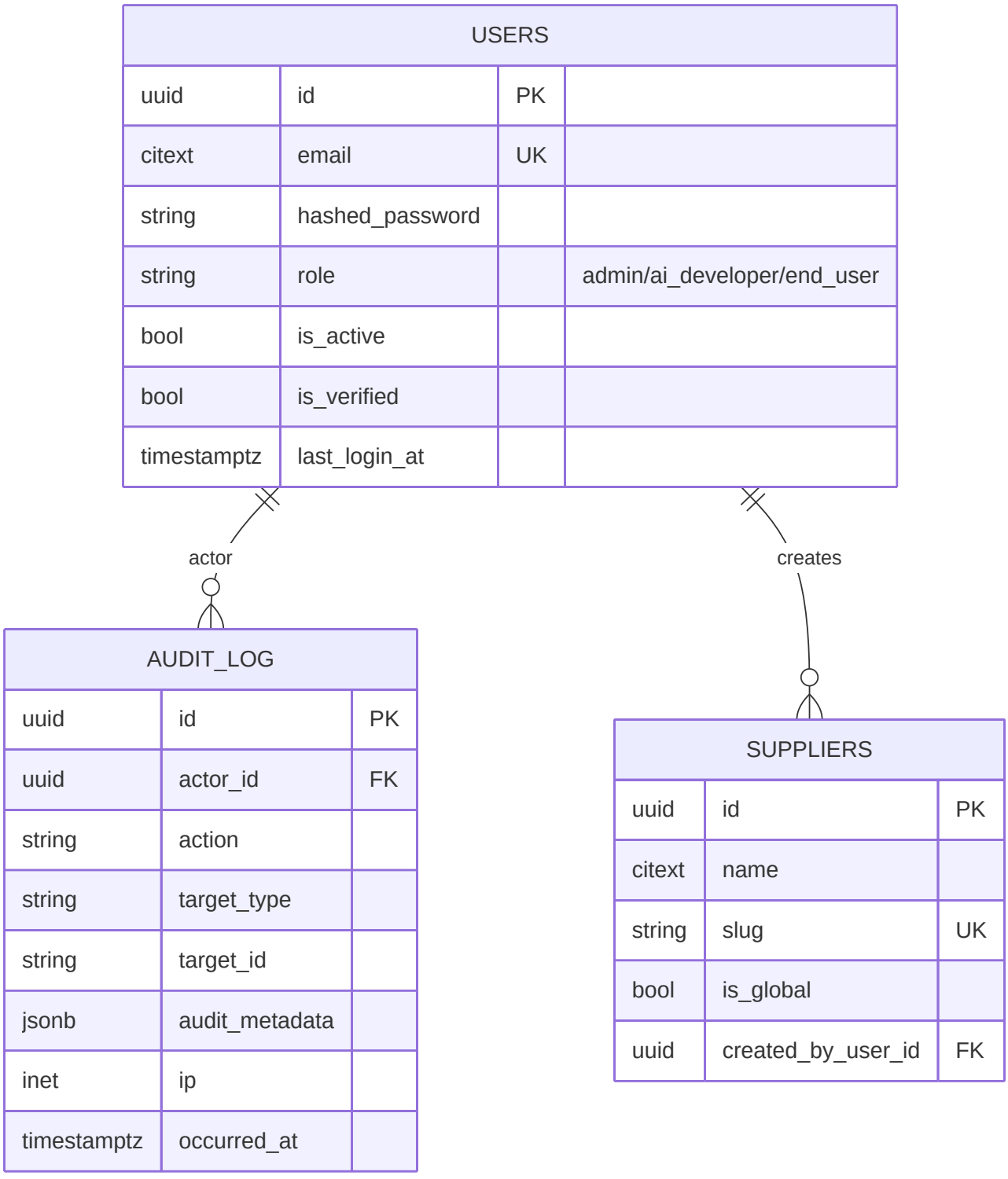

Figure 4.9: Database schema: audit trail and the suppliers a user creates.

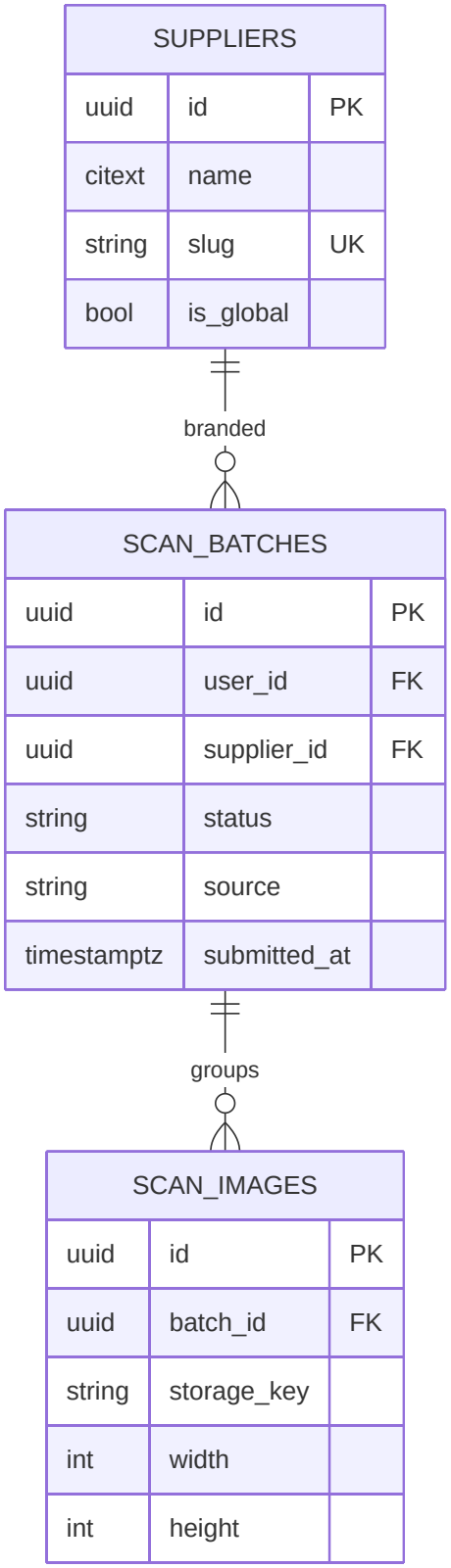

Figure 4.10: Database schema: scan batch intake — suppliers, batches, and images.

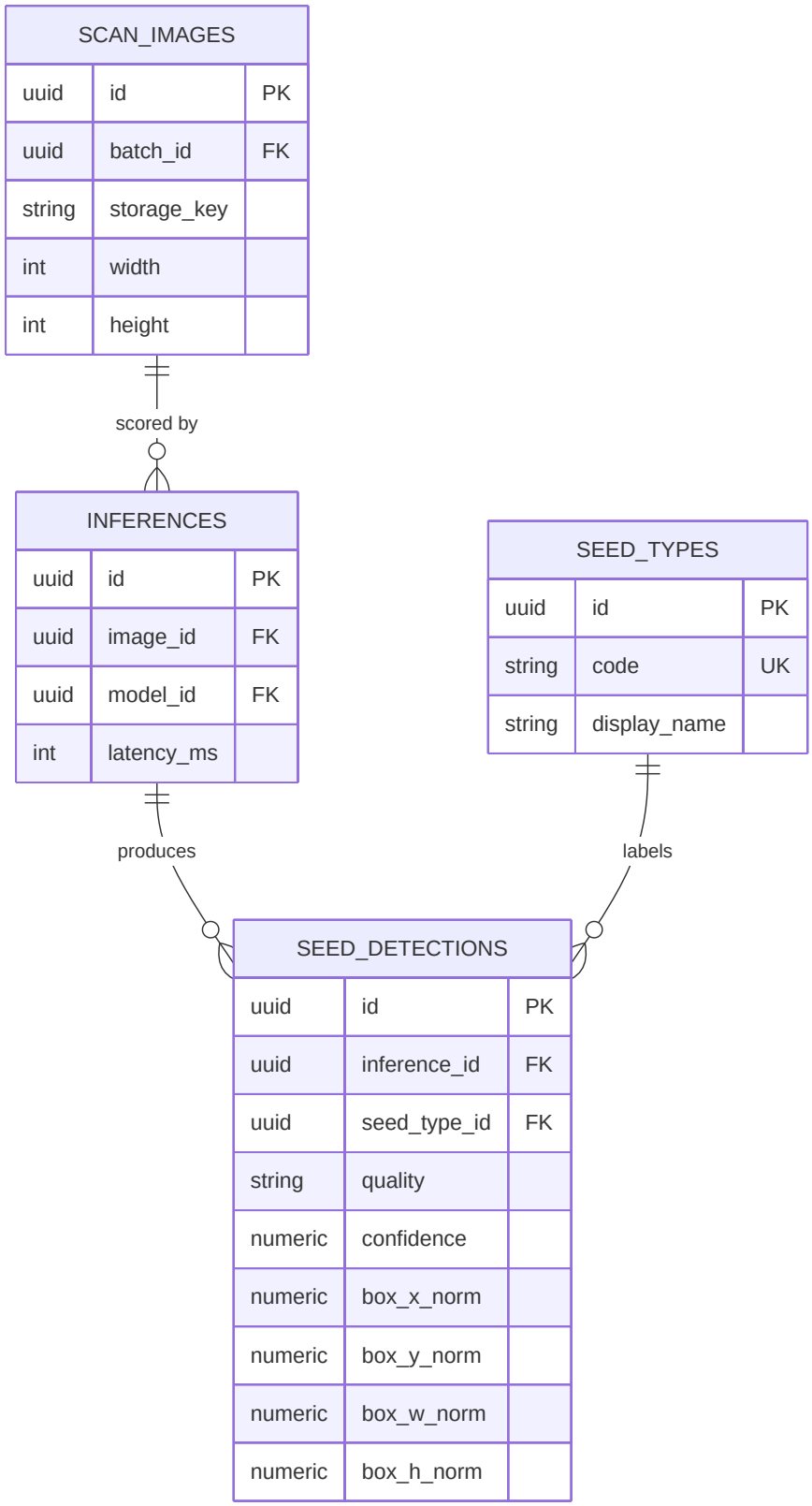

Figure 4.11: Database schema: inference results — images, model inferences, and seed-level detections.

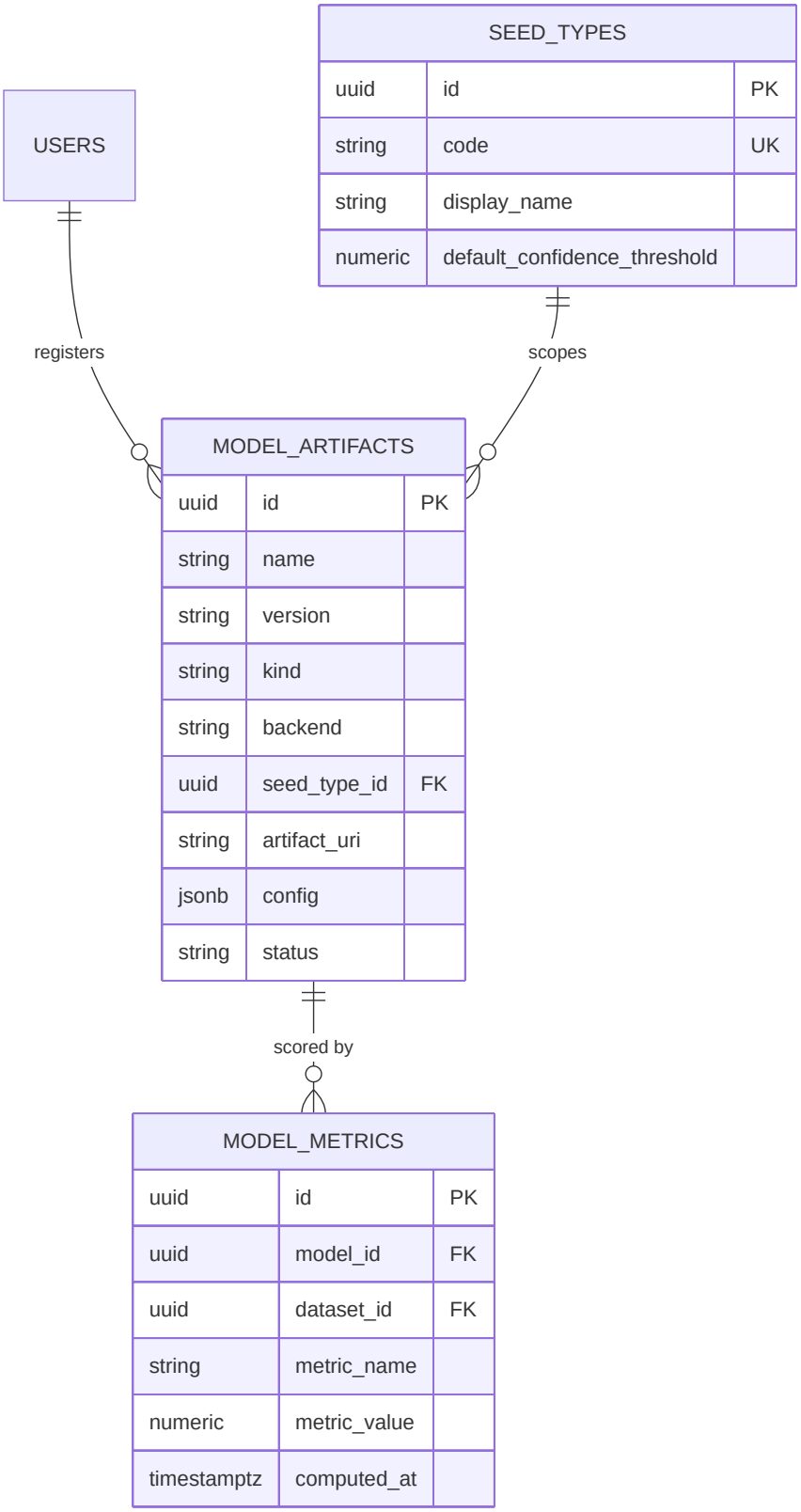

Figure 4.12: Database schema: model registry — versioned artifacts scoped by seed type and scored via performance metrics.

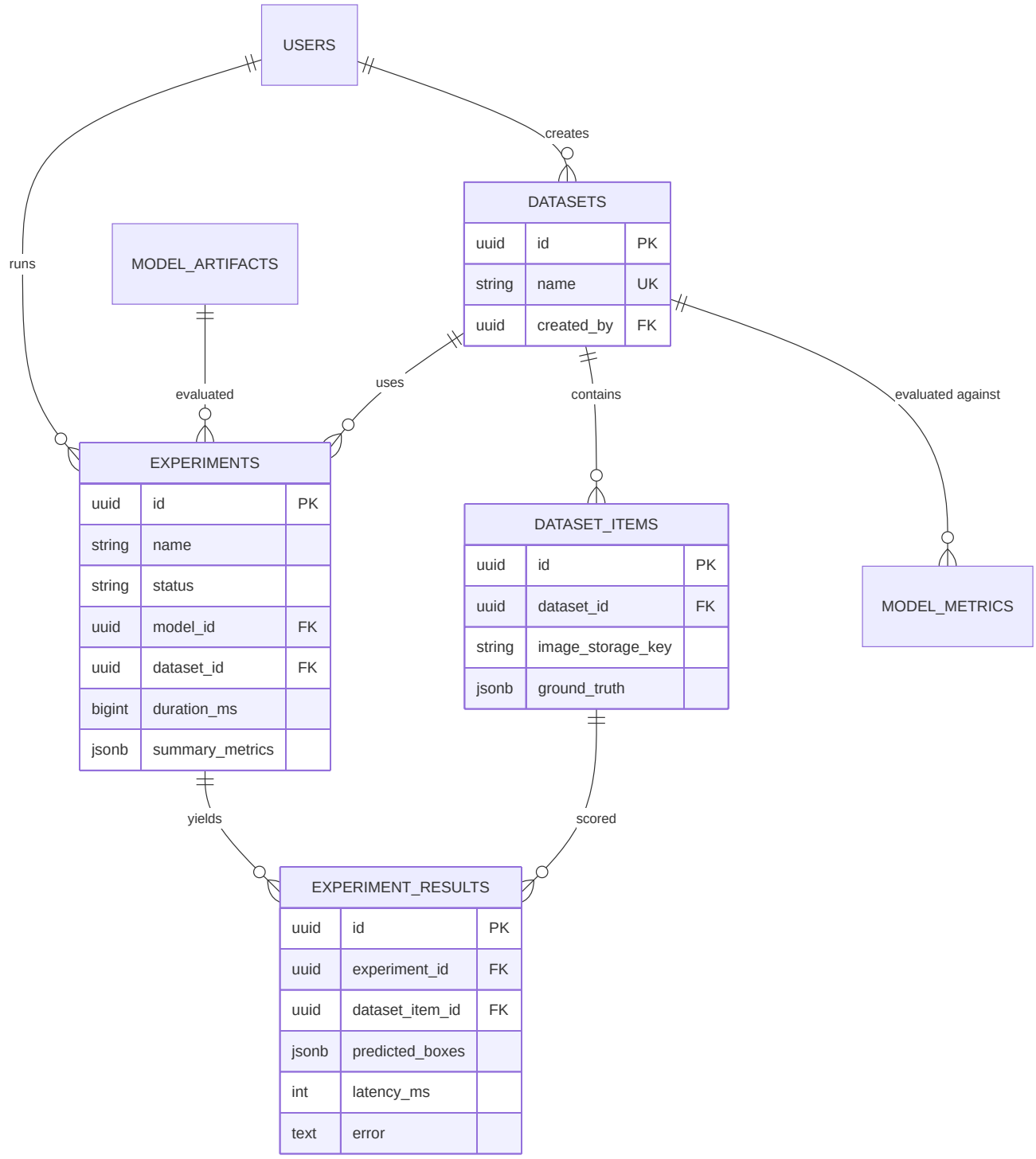

Figure 4.13: Database schema: offline evaluation — labelled datasets, experiment runs, and per-item results against registered models.

4.5 MultiSeedGen Design

Training a seed detector needs many photographs of cluttered, overlapping seeds with a correct bounding box on every seed. These are slow to capture and tedious to annotate by hand. MultiSeedGen is the companion tool that manufactures this data. It takes photographs of single seeds and composites them into annotated multi-seed scenes whose labels are correct by construction, an instance of cut-and-paste synthesis [3].

The pipeline begins by segmenting each seed out of its source photo into a clean cut-out. A classical colour-and-brightness method is tried first, and a learned matting model is the fallback, either U2-Net [14] or Segment Anything [15]. The clean cut-outs form a seed pool.

Scene composition then paints seeds onto a background. It samples a number of seeds per scene and places each one with controlled placement, rejecting any position that overlaps an existing seed too much or that leaves a seed too hidden to label. Each placed seed gets its own augmentation (scale, rotation, and colour jitter) chosen so it does not invalidate the label. The seeds are composited onto varied backgrounds, from solid and textured fills to real inpainted photos. Some camera-style degradation (blur, noise, and compression) is added so the synthetic images resemble real phone photos, a form of domain randomization that helps a model trained on them transfer to real input [4], [16].

Because the tool knows exactly where it placed each seed, it records the exact bounding boxes and masks for every scene, accounting for seeds partly hidden behind others. Export then writes the dataset in the formats a trainer expects: YOLO boxes and segmentation polygons, and COCO annotations [17]. A leakage-safe split makes sure a source seed reserved for validation never appears in any training image, so the validation score is not inflated by the same seed showing up on both sides. The resulting detection and segmentation datasets are what train the Seed Bank detector.

4.6 User Interface Design (Wireframes and Mockups)

The prototype front end is a React web app that walks the user through the analysis workflow, from the dashboard to the per-seed results and the saved history. The screens below are the main views.

4.6.1 Dashboard

The dashboard is the central entry point for the workflow, shown in Figure 4.14. It presents a prominent drag-and-drop zone for uploading images, a few headline figures, and a list of recent scans, with a status indicator that confirms the backend is reachable.

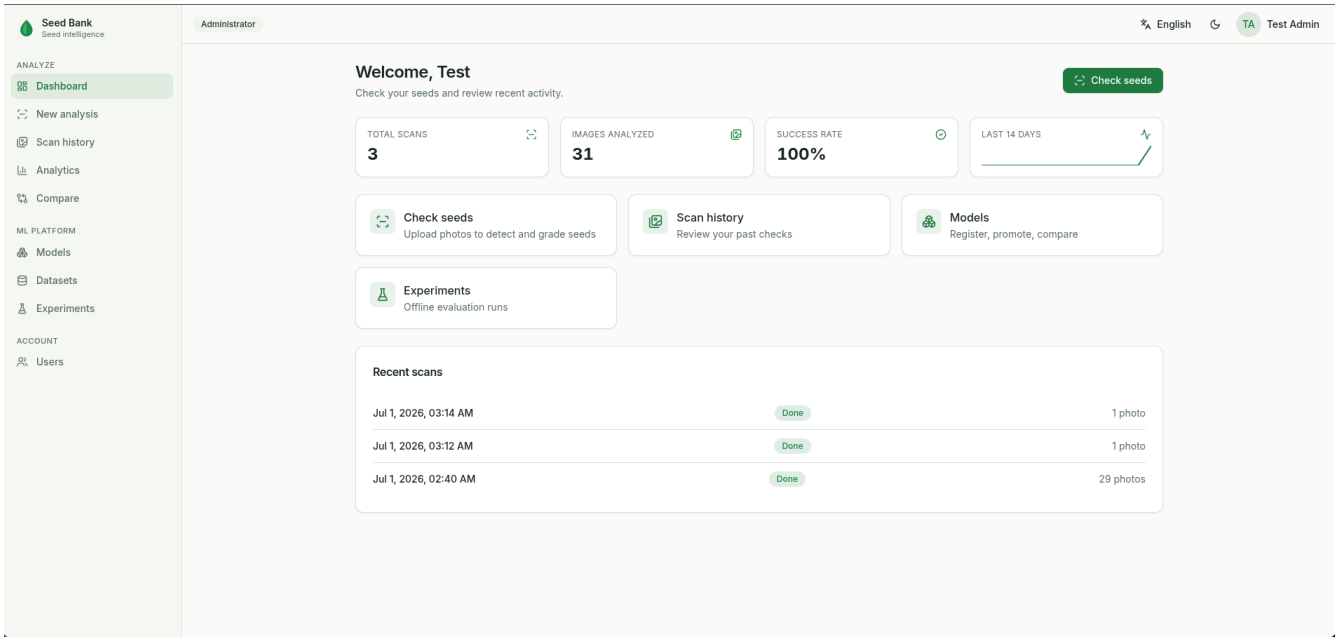

Figure 4.14: Dashboard: the drag-and-drop upload zone, headline figures, and recent scans.

4.6.2 Analysis and results workspace

The analysis results workspace is where the user verifies the predictions, shown in Figure 4.15. The processed image is displayed with color-coded bounding boxes, green for good seeds and red for bad seeds, so the overall quality reads at a glance. A panel alongside the image gives aggregate metrics, including the total detection count and the purity percentage. Below it, a scrollable per-seed list details each detection with its confidence score. A tabbed bar lets the user move between the images in a multi-image batch without losing context.

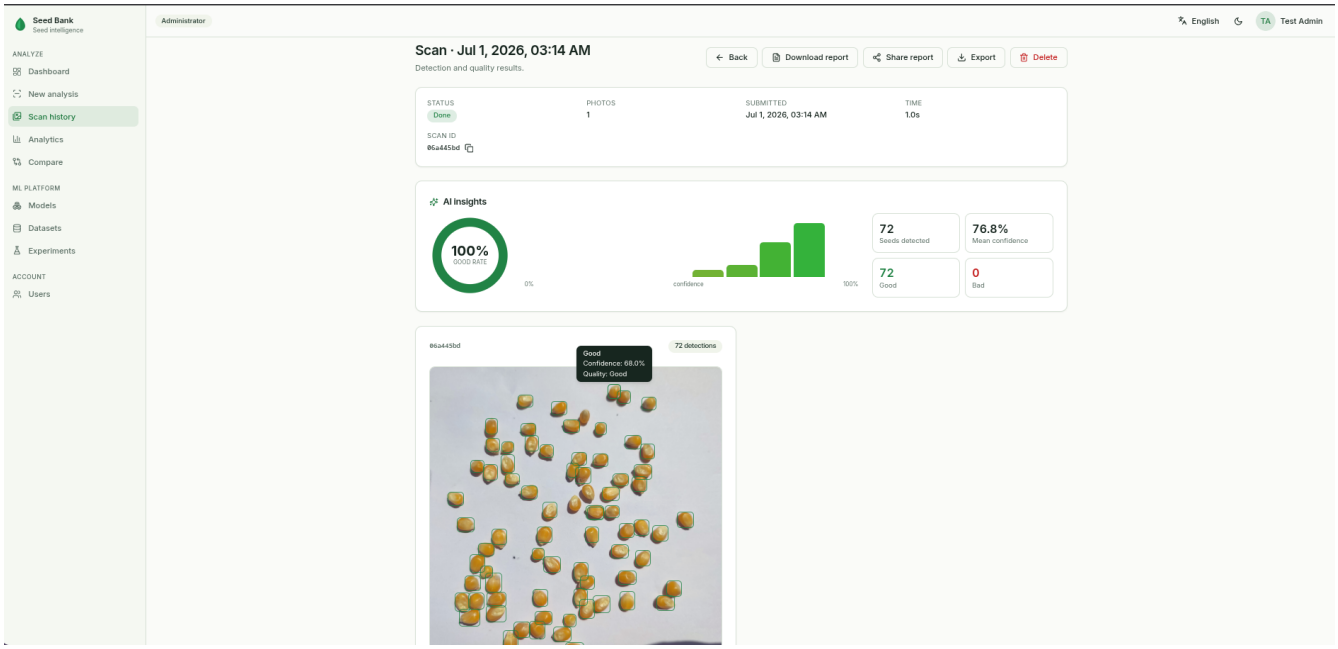

Figure 4.15: Analysis and results workspace: color-coded bounding boxes (green good, red bad), the per-seed list, and the purity panel.

4.6.3 Reporting and export

The reporting view generates an exportable session summary, shown in Figure 4.16. It opens with high-level metrics, including a good versus bad seed count shown as a pie chart and a per-image breakdown table that points out skewing samples. Each image is detailed with its annotated bounding boxes, and a final table lists every detected seed with a thumbnail, its quality label, and its confidence score, so a reviewer can check the results seed by seed.

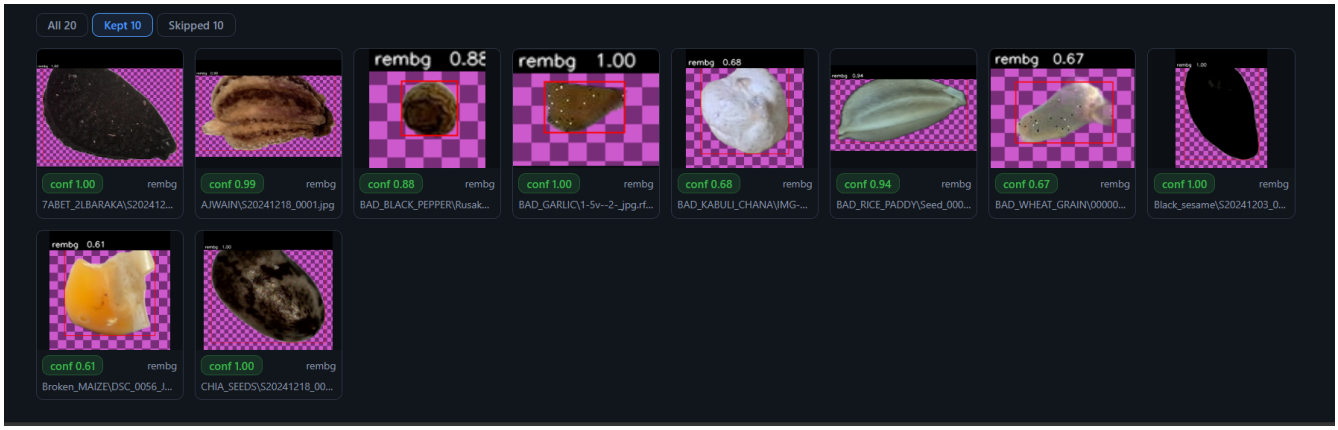

Figure 4.16: Reporting and export view: high-level metrics, the good-versus-bad pie chart, the per-image breakdown, and the per-seed table.

4.6.4 Scan history

The scan-history view is the repository for previously run analyses, shown in Figure 4.17. A statistical panel at the top aggregates global figures, such as the total number of batches processed and the cumulative seed count. Below it, each past session is a chronological card showing the timestamp, the final status, and a good-versus-bad breakdown with the average confidence. A filter sorts records by status, and a ”View Details” action reopens the full visual data and report for any archived session.

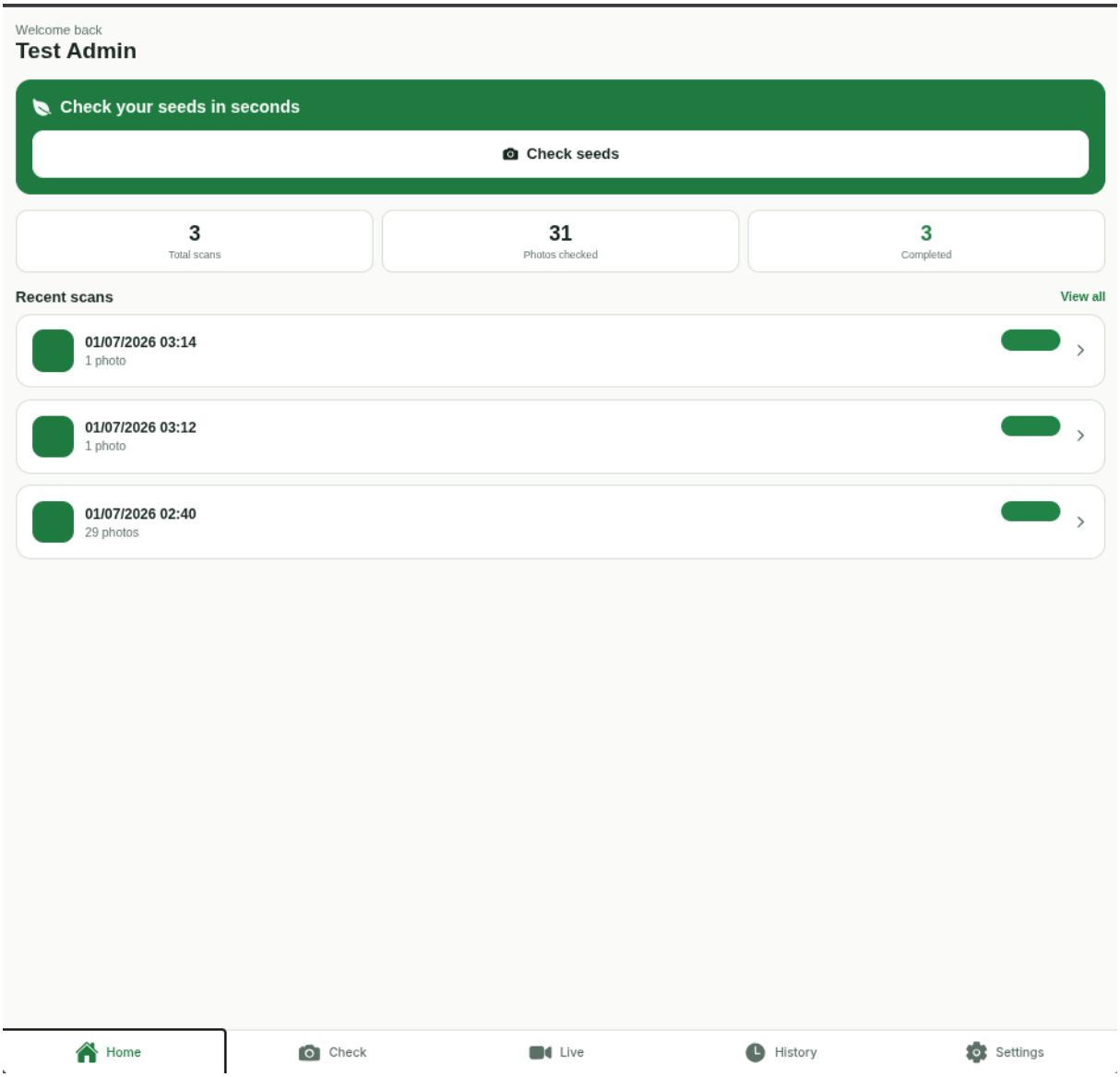

Figure 4.17: Scan history: the global statistics panel and the chronological session cards with filtering and per-session detail.

4.7 Security Design Considerations

Security is brief and standard for a prototype of this size. Sessions use tokens: the user signs in and the backend issues a short-lived access token that the client sends with each request. Passwords are never stored in plaintext; only their hashes are kept, so a database leak does not expose a usable credential. Access is role-based, so an ordinary user can analyze and see only their own history, while administrative and model-management actions are gated to the roles that are allowed to perform them.

Chapter 5

Implementation

The previous chapter described what the system should do and how its pieces fit together. This chapter is about how those pieces were actually built: the setup that lets the whole platform run on a laptop, the implementation choices that turned the design into working code, and a few screenshots and generated samples that show the result. The synthetic-data tool, MultiSeedGen, gets the most space, because it is where most of the engineering effort went and it is the part of the project we can defend in the most detail.

5.1 Implementation environment and setup

The whole system is containerized with Docker [7]. The goal was that a fresh checkout reaches a working stack with a single command, without anyone installing a database or a Python toolchain on the host by hand. Docker gives us an environment that behaves the same on a personal laptop, a lab machine, or a server, which removes the usual ”it works on my machine” problem and makes it easy for a new team member to get started.

There are four primary components. The frontend is a React application [6] that the user interacts with. The backend is a FastAPI service [5] that takes an image, runs the models, and returns the results. PostgreSQL stores the transactional records so a user can come back later and see the history of what they scanned. Alongside PostgreSQL, a ClickHouse instance acts as the analytics data warehouse, storing detailed inference telemetry and performance metrics for reporting and evaluation. The frontend talks to the backend over a simple HTTP API, and the backend communicates with the databases and object storage, so each part can be developed and tested on its own.

The build view is split into two: Figure 5.1 shows the multi-stage Dockerfile’s 4 builder stages feeding their 4 matching runtime stages; Figure 5.2 shows which runtime stage each of the three Compose images is built from. The runtime view is split into three: Figure 5.3 shows the three containers the default profile runs; Figure 5.4 shows those same containers wired to the stateful datastores; Figure 5.5 shows the same containers alongside the opt-in dev/obs/frontend

tooling profiles.

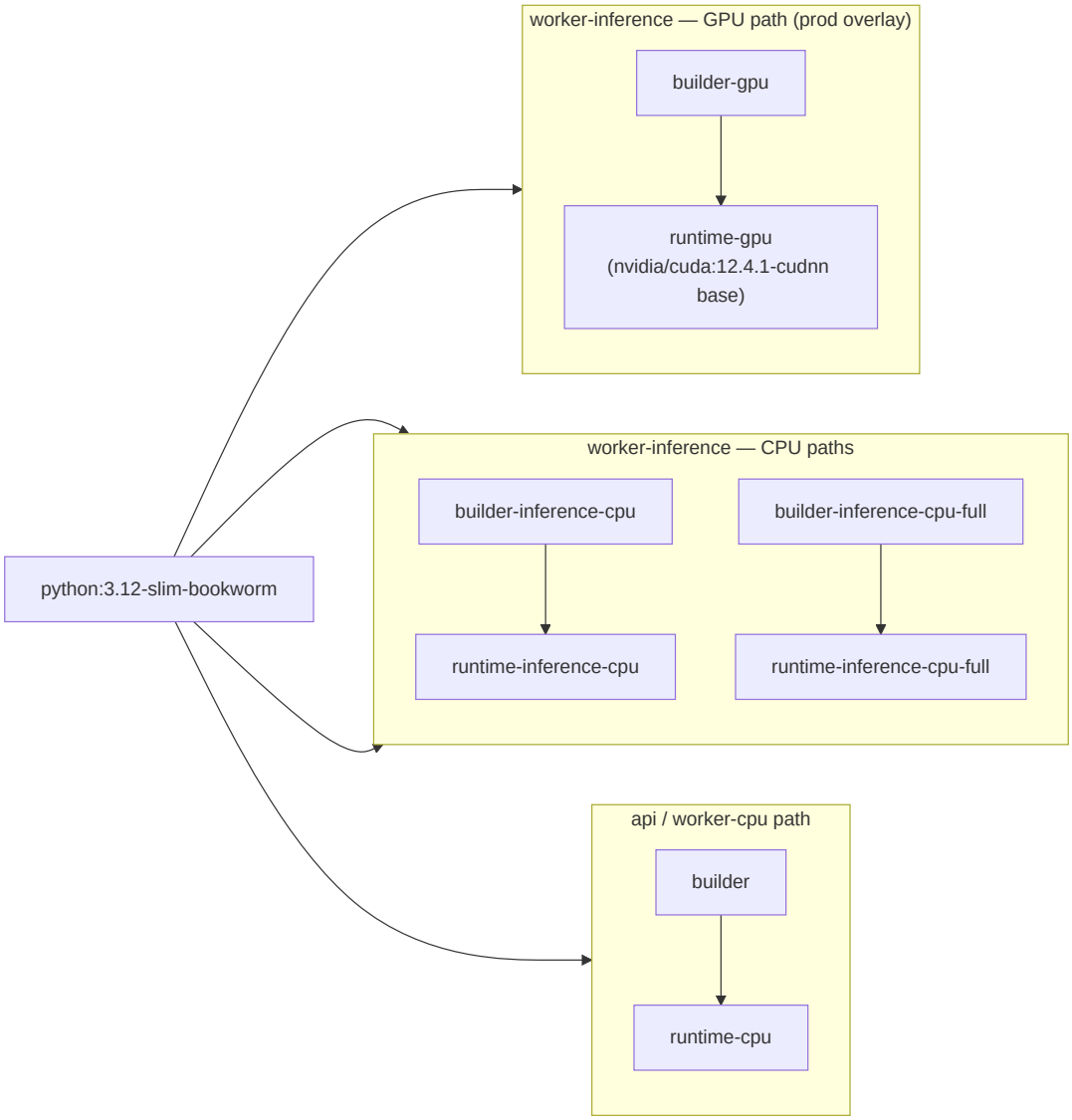

Figure 5.1: Build view: the multi-stage Dockerfile's 4 builder stages feeding their 4 matching runtime stages.

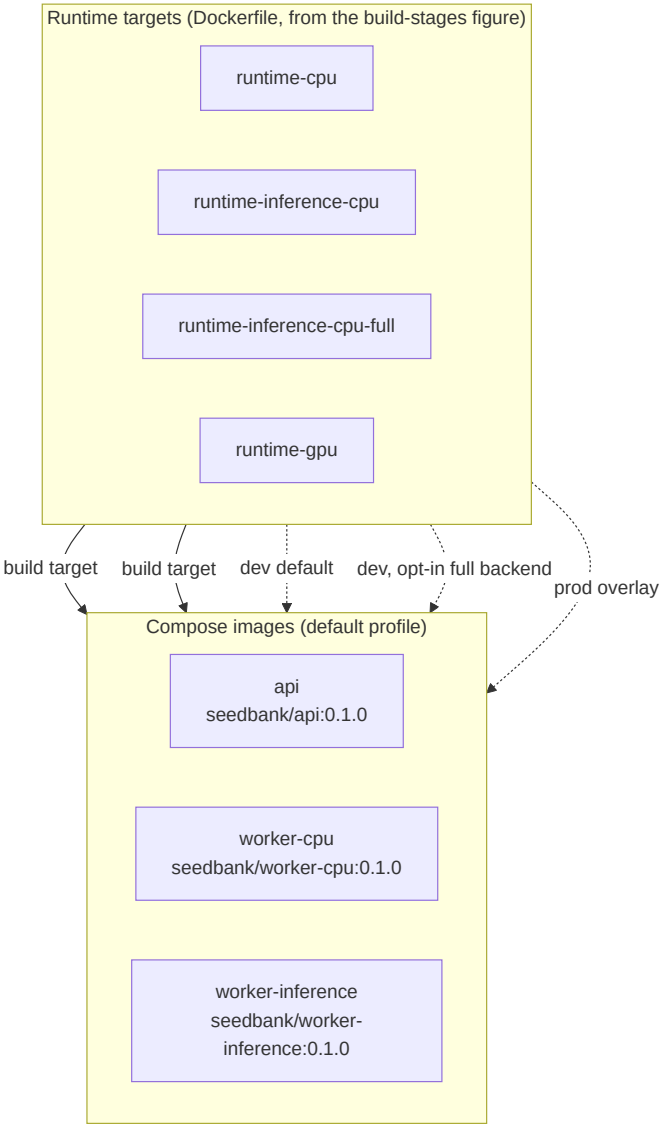

Figure 5.2: Build view: the three Compose images (`api`, `worker-cpu`, `worker-inference`) and which Dockerfile runtime stage each is built from.

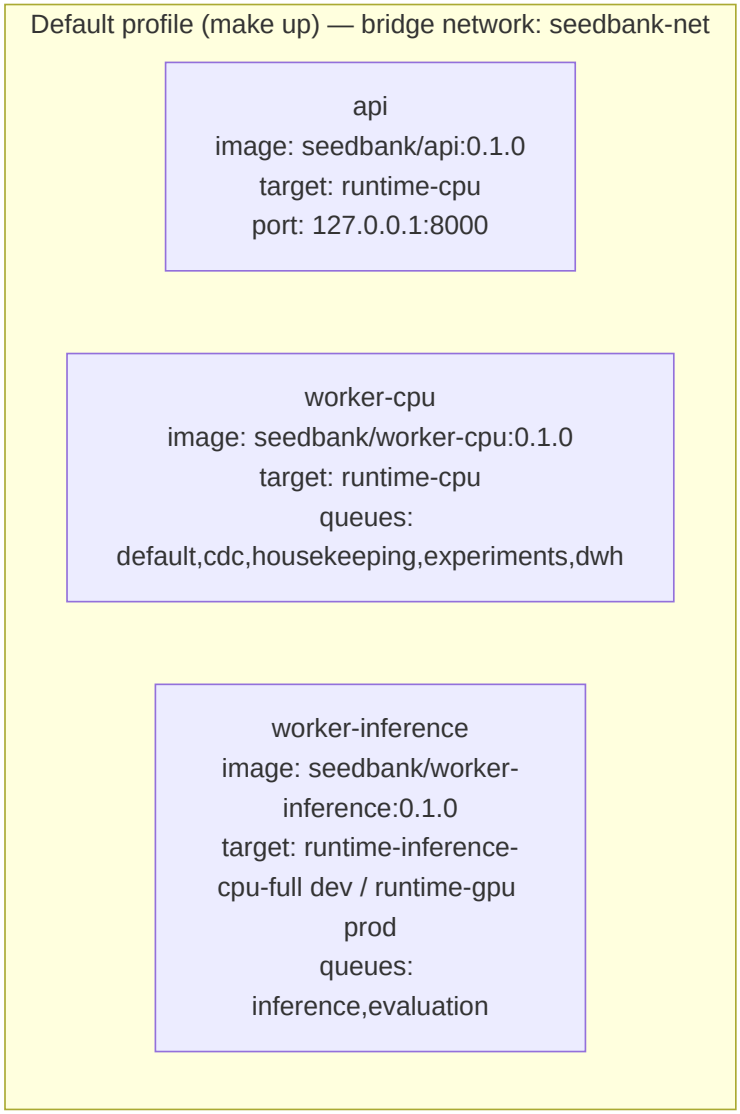

Figure 5.3: Runtime view: the `api`, `worker-cpu`, and `worker-inference` containers the default Compose profile runs, all on the `seedbank-net` bridge network.

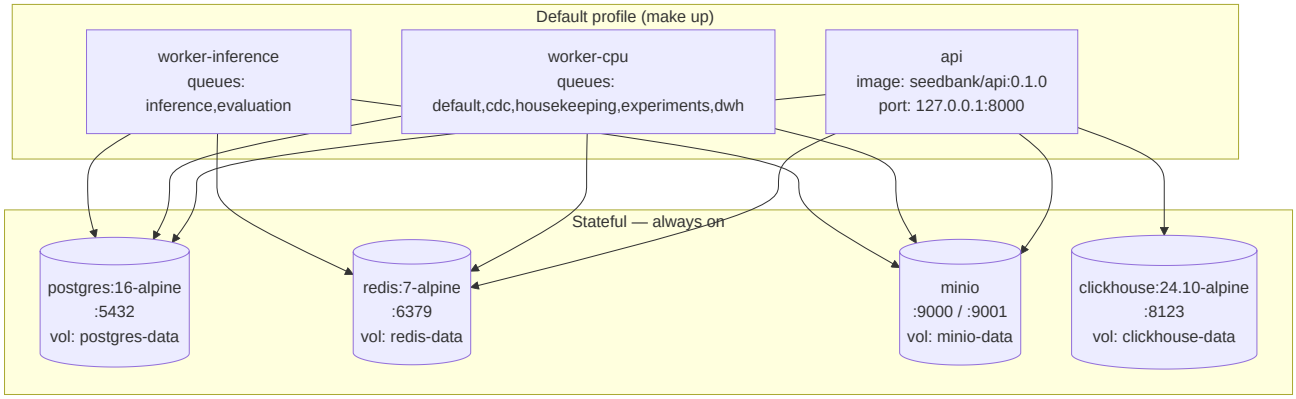

Figure 5.4: Runtime view: the three containers wired to the stateful Postgres/Redis/MinIO/ClickHouse datastores.

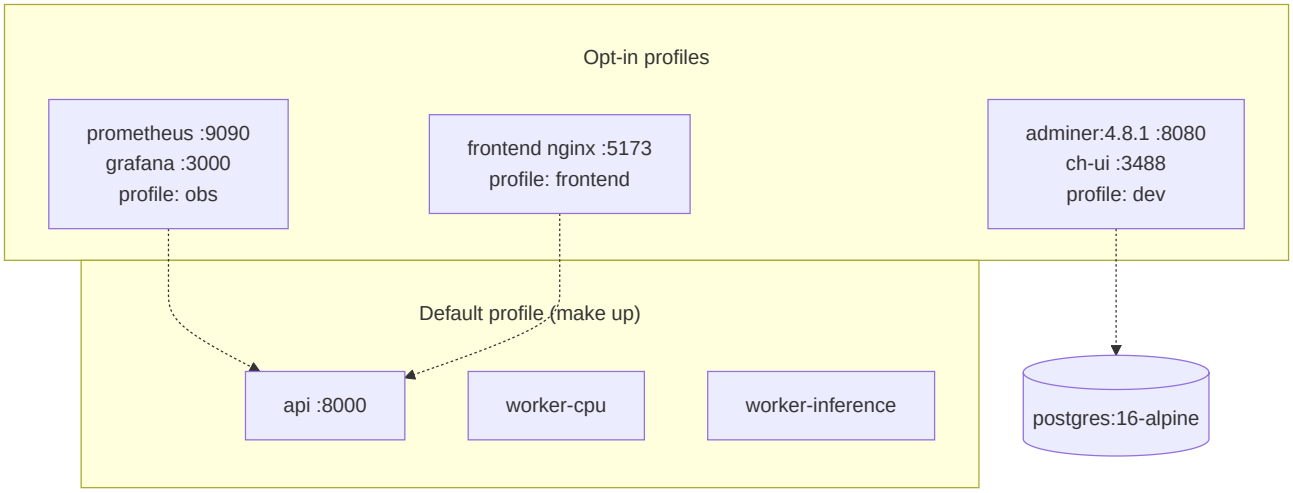

Figure 5.5: Runtime view: the three containers alongside the opt-in dev (Adminer/ch-ui), obs (Prometheus/Grafana), and frontend tooling profiles.

5.2 Key implementation details

5.2.1 Computer-vision background

The computer vision pipeline is built to solve two distinct problems. The first is an inter-class problem: separating different seed types from one another within a mixed photograph. The second is an intra-class problem: addressing the quality differences between individual seeds of the same type. These correspond to object detection and per-seed classification, respectively. The following sections describe the model families used for each stage, leaving the detailed implementation and results to later chapters.

5.2.2 Two-stage detectors

Our detection model uses a ResNet backbone to extract strong visual features from the image, while the FPN and PANet help combine information from different feature levels so the model can detect both small and large objects better. In Faster R-CNN, the Region of Interest (ROI) part selects important areas from the image and classifies them more accurately. During training, CIoU is used to improve bounding box regression by making the predicted box overlap, center distance, and aspect ratio closer to the ground truth. Together, these components make the model more accurate and reliable for object detection. ResNet backbone is the feature extractor at the start of the model. It takes the image and learns useful patterns like edges, shapes, and textures, so the detector has a strong representation of what is in the image.

FPN (Feature Pyramid Network) helps the model use features from different scales. This is important because small objects need fine details, while large objects need broader context, so FPN combines information from multiple levels of the network.

PANet (Path Aggregation Network) improves on FPN by strengthening the flow of information between low-level and high-level features. In simple terms, it helps the model pass details more effectively so detection becomes better, especially for objects of different sizes.

ROI (Region of Interest) refers to the important areas in the image that the detector focuses on after proposals are made. In Faster R-CNN, these regions are cropped and analyzed more carefully so the model can classify the object and refine its box.

CIoU (Complete Intersection over Union) is a loss function used to make bounding box predictions better. It does not just compare overlap, but also considers the distance between box centers and the box shape, which makes localization more accurate.

Faster R-CNN architecture is a two-stage object detection model. First, it finds possible object regions, then it classifies them and adjusts the boxes more precisely, which makes it accurate and reliable, though usually slower than one-stage models like YOLO.

Figure 5.6 shows the shared feature-extraction backbone; Figure 5.7 shows the detection stage that consumes those feature maps to produce the final class label and bounding box.

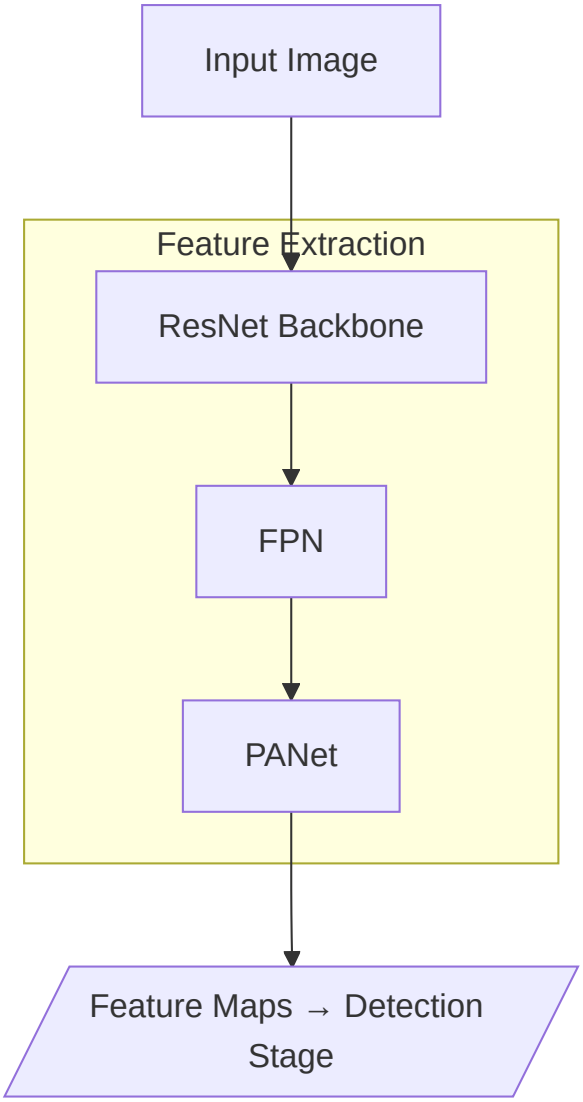

Figure 5.6: The Faster R-CNN feature-extraction backbone, showing the ResNet backbone feeding the FPN and PANet to produce the multi-scale feature maps passed to the detection stage.

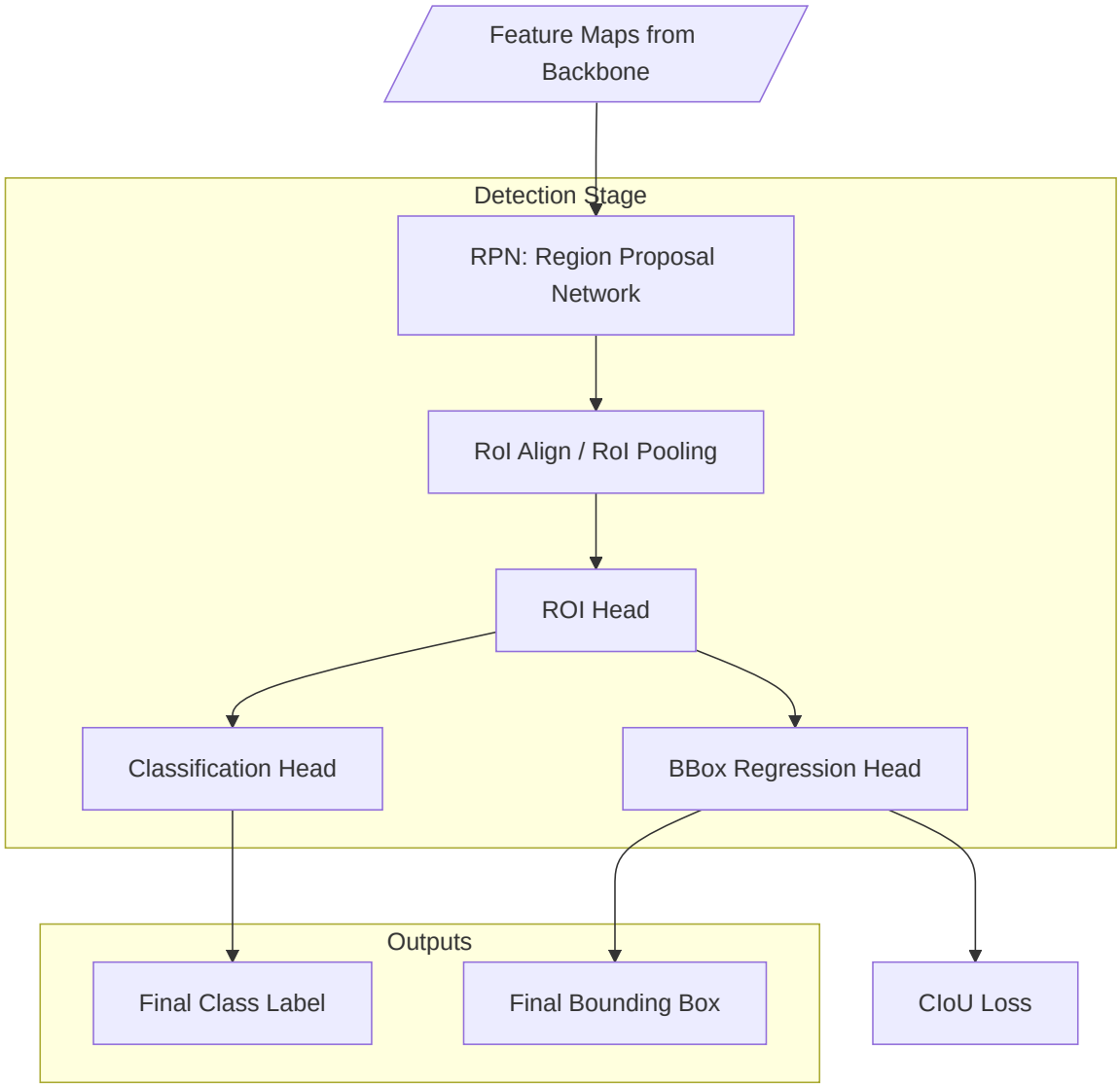

Figure 5.7: The Faster R-CNN detection stage, showing the RPN and ROI head consuming the backbone's feature maps to produce the final class label and bounding box.

The second stage is quality classification. To perform multi-label defect classification on individual isolated crops, the pipeline utilizes a deep convolutional neural network optimized for fine-grained feature extraction. Rather than relying on a standard off-the-shelf classifier, the architecture is a modified variant of EfficientNet-B2 integrated with a Convolutional Block Attention Module (CBAM) and a hybrid pooling layer.[18]

The system leverages an ImageNet-pretrained EfficientNet-B2 to serve as the primary feature extractor, CBAM Attention Layer is inserted immediately after the final feature map of the backbone. CBAM applies a sequential dual-attention mechanism.

Channel Attention determines what defect characteristics to emphasize across channels, while Spatial Attention isolates where those defects are located on the physical seed geometry. Crucially, this attention block is wrapped in a residual connection to refine existing backbone features without degrading the network's underlying gradient stream.

Hybrid Pooling Head is the standard global average pooling layer is replaced with a concurrent, hybrid concatenation layout. By running Global Average Pooling (GAP) and Global Max Pooling (GMP) in parallel and splicing their outputs the network simultaneously tracks smooth structural global patterns alongside highly localized, sharp defect signals (such as microscopic fungal patches or fine cracks).

5.2.3 One-stage detectors

As a prominent class of one-stage architectures, single-shot detectors bypass the traditional region proposal phase entirely, enabling the simultaneous prediction of bounding box coordinates and corresponding class probabilities within a single forward pass. This unified approach was pioneered by YOLO [19], which reformulated the object detection problem as a singular regression task mapped across a global grid framework. While this design strategy exchanges a marginal degree of spatial localization precision for a massive optimization in computational speed, it also yields strong generalization properties. Because the network processes images globally, it effectively learns robust representations that resist overfitting to localized environmental noise, thereby facilitating smoother transfer learning and superior domain adaptation when trained on synthetic datasets.

5.2.4 The two-stage detection and classification pipeline

Checking a photo of seeds is really two separate problems, and the implementation treats them as two models run one after the other. The first stage is detection. An object detector scans the whole image and returns, for every seed it finds, a bounding box, a confidence score, and the crop type, so the detector tells us where each seed is and which of the two it is. One photo therefore becomes a set of located, labelled seeds.

The detector is a Faster R-CNN [20] with a ResNet-50 backbone [21] and a Feature Pyramid Network [22]. The feature pyramid lets the model find seeds at different sizes in the same image, which matters because a photo can hold a hundred seeds at slightly different distances from the camera. Faster R-CNN is the accurate option, while the second architecture, PANet, further strengthens feature flow across scales to improve detection of small and overlapping seeds. For situations where speed matters more than the last point of accuracy, such as a conveyor-belt feed or a phone in the field, there is an optional YOLO mode [19], [23] that runs the detection in a single pass and is much faster, at some cost in precision.

The second stage of the pipeline is fine-grained, multi-label classification. For each isolated seed, the system extracts the bounding box crop from the original image and routes it to a specialized quality network, which outputs predictions across a vector of concurrent defect conditions alongside their respective confidence scores. The core architecture leverages an ImageNet-pretrained EfficientNet-B2 backbone [18], selected for its optimal depth-width-resolution scaling balance and high computational throughput when processing dense seed batches.

5.2.5 Model management and evaluation

The backend also has a small amount of supporting machinery around the models. It can register and swap the trained weights without code changes; it can run an offline evaluation of a model against a held-out set, storing each run's metrics in PostgreSQL alongside a Markdown report in object storage so that versions can be compared before one is put into use; and it runs the actual inference in a background worker so that uploading a batch of images does not block the rest of the application while the models run. This part stayed simple on purpose, since the prototype only needs to serve and compare a handful of models.

Choosing which model actually runs for a given request is the job of the model resolver. Given a segment the kind of model needed (detection or classification) and, optionally, the seed type it resolves the model promoted to `production` for that exact segment; if none has been promoted and a seed type was supplied, it falls back to the global, seed-type-agnostic production model for that kind; and if nothing is routable it reports that no model is ready rather than failing silently. The decision is deterministic and stateless there is no weighted routing and no per-user bucket so the same request always resolves to the same model, and a developer can override the choice with an explicit model on a single request. Figure 5.8 shows the decision.

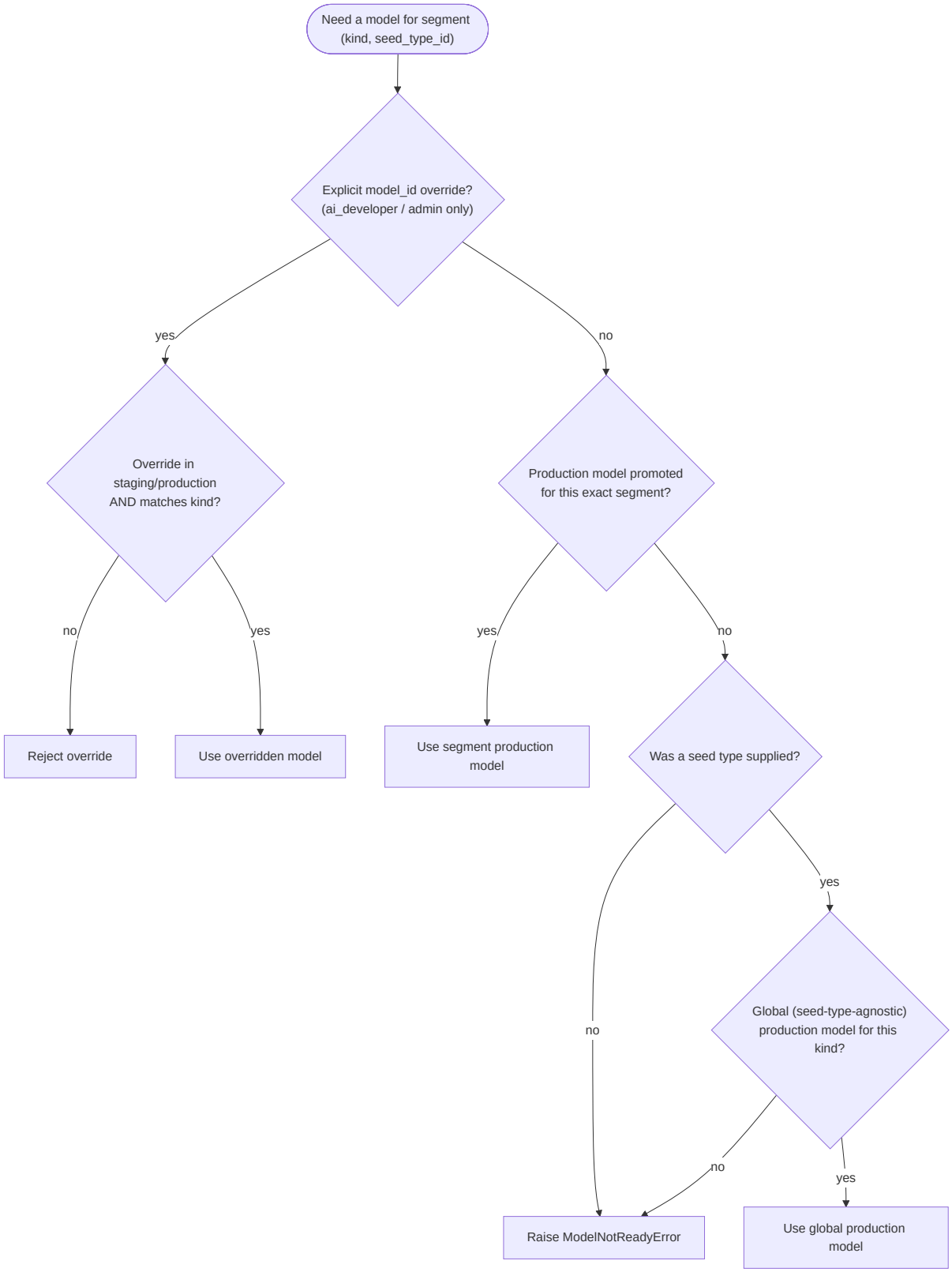

Figure 5.8: Model resolution: the resolver selects the model promoted to production for a segment, falls back to the global production model when only that is available, or reports that no model is ready.

5.2.6 Data warehouse and analytics telemetry (ClickHouse)

To track the real-world performance of the grading system, the platform implements an online analytical processing (OLAP) star schema in ClickHouse. While PostgreSQL manages transactional and relational data (such as user credentials, scan batches, and model status), ClickHouse is dedicated to high-performance analytics, storing scan telemetry, inference latency, model accuracy over time, and geographic distribution metrics.

The analytics database schema consists of dimension tables (such as `dim_user`, `dim_model`, and `dim_seed_type`) and fact tables (such as `fact_inference`, `fact_detection`, and `fact_scan_batch`). Data is synchronized from PostgreSQL using application-level dual-writes rather than log-based change data capture (CDC). After each successful transactional commit, a Celery worker dispatches an idempotent insert to ClickHouse via the `AnalyticsRepository` using the async driver `clickhouse-connect`.

Because ClickHouse uses the `ReplacingMergeTree` table engine, duplicate inserts collapse on their sort key during merge cycles. This makes at-least-once delivery from the Celery tasks completely safe against duplicates, ensuring high consistency without complex distributed locking. Callers query this database on async read paths to drive the dashboard statistics, leaderboards, and offline performance tracking.

5.2.7 MultiSeedGen: the synthetic-data pipeline

Training a detector needs labelled images that contain many seeds at once, with a box around each one. Those are far harder to collect than the single-seed photos in the public datasets: someone would have to lay out dozens of seeds, photograph them, and then hand-draw and label a box around every seed in every image. MultiSeedGen avoids that work. It cuts individual seeds out of single-seed source photos, pastes many of them onto realistic backgrounds, and exports a fully labelled multi-seed detection dataset. Because the tool places every seed itself, it already knows exactly where each seed is and what shape it has, so the bounding box and the segmentation mask are known precisely. The annotations come for free, with no manual labelling.

The hardest step is segmentation: cutting one seed cleanly out of its source photo, including the soft edges. MultiSeedGen runs this as a cascade that starts cheap and escalates only when needed. It first tries classical methods, a colour-distance threshold against the background followed by GrabCut, which are fast and good enough for seeds on a plain background. When those produce a poor mask, it falls back to rembg, a U^2-Net model [14], and for the hardest cases with feathered or low-contrast edges it uses Segment Anything [15]. A cut-out whose mask scores too low is dropped rather than pasted in with a bad outline, so a poor segmentation never becomes a bad label.

With clean cut-outs in hand, the tool composites a scene. It pastes many seeds onto a background, the cut-and-paste idea from Dwibedi et al. [3], varying the backgrounds so the detector

does not just learn one surface. To stop the synthetic images from looking too clean and too similar to each other, MultiSeedGen applies domain-randomized augmentation: it randomizes the geometry of each seed, its rotation, scale, and placement, and the lighting and colour of the scene [4], [16], [24]. The aim is for a model trained on these images to transfer to real photos, since it has seen a wide range of arrangements and conditions rather than one narrow style.

Two details keep the resulting dataset trustworthy. First, the split between training and validation is leakage-safe: a given source seed is assigned to either training or validation, never both, so the same physical seed cannot show up in a training image and a validation image at once. Without this, the validation score would be inflated, because the model would effectively be tested on seeds it had already seen. Second, the datasets export to both COCO and YOLO formats [17], the two formats the detection models expect, so the generated data drops straight into training.

5.3 Sample Screenshots and Output Samples

This section shows the running system and the results it produces across its various modules, encompassing data generation, real-time inference, and analytical reporting.

The end product of the pipeline during live evaluation is an annotated image overlay. Figure 5.9 shows the real-time detection output drawing bounding boxes directly over the target frame, with each seed assigned a granular multi-label classification (e.g., Shriveled, Damage, or Good). This is the continuous overlay the frontend shows the user during an active scan.

Figure 5.9: Annotated real-time output: detection boxes dynamically drawn with granular per-seed defect labels generated by the classification model.

To train these highly accurate detection models, synthetic data is utilized. Figure 5.10 is a composite scene generated by the MultiSeedGen tool, showcasing the dynamically calculated

bounding boxes drawn over the exported synthetic labels. This illustrates the highly varied, lab-like imagery the tool produces by the thousands to train the detector.

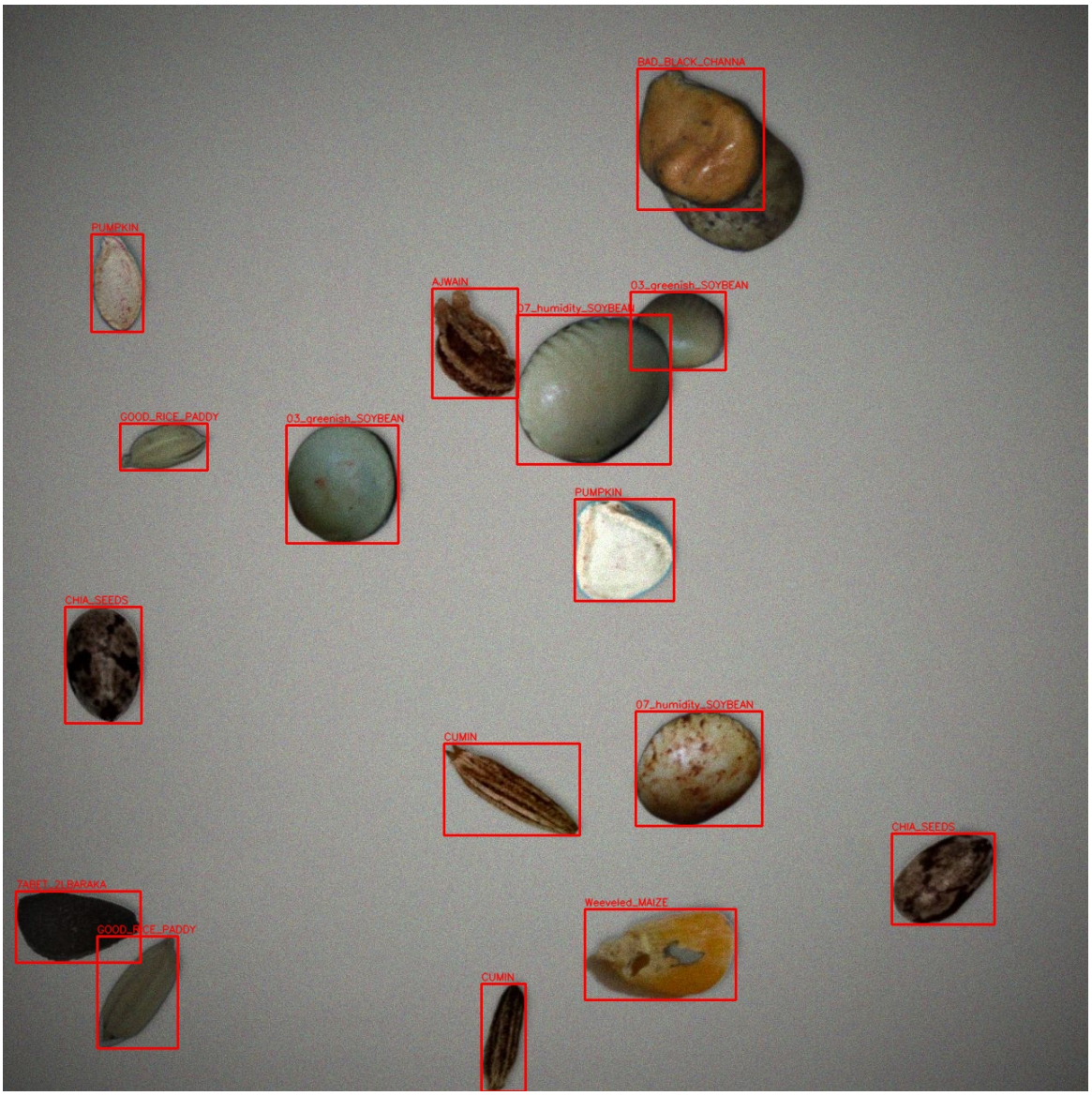

Figure 5.10: A MultiSeedGen synthetic scene: many augmented, cut-out seeds seamlessly composited onto a tray background, annotated with accurate bounding boxes.

Prior to generating these massive datasets, the extraction parameters must be verified. Figure 5.11 is the segmentation tuner from the interface, which displays the kept and skipped seed cut-outs side by side. This allows the user to visually inspect which seeds a given threshold dropped, and why, before executing a full-scale generation run.

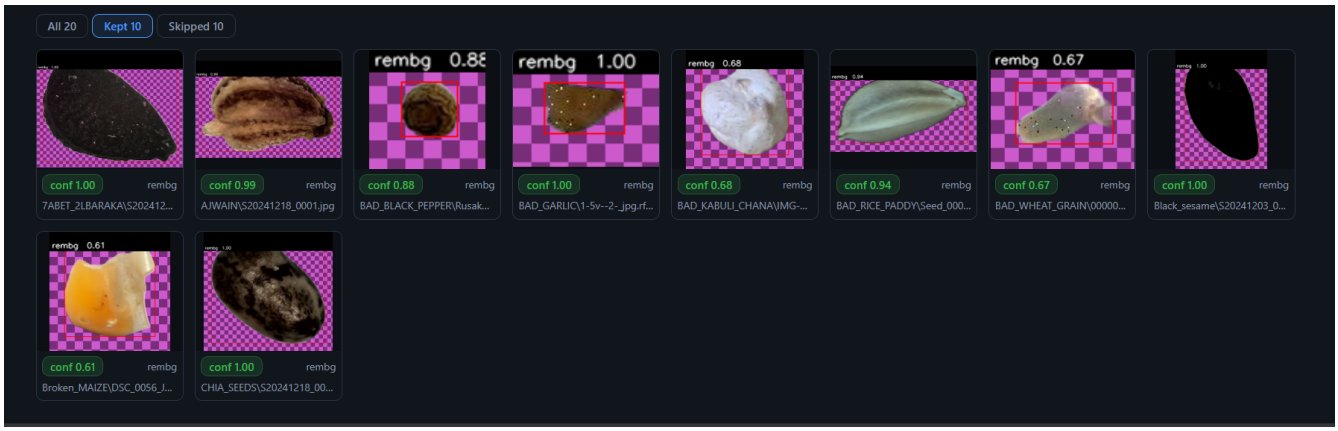

Figure 5.11: Segmentation tuner interface: kept and skipped cut-outs are displayed together, enabling the user to evaluate and adjust segmentation quality thresholds.

Once real-world evaluation begins, the end product of processing an entire collection sample is the batch classification summary. Figure 5.12 illustrates the structured batch report generated after scanning dense arrays of seeds, tracking unique filenames, file previews, and their total detected element counts. This view allows users to manage and verify bulk scanning tasks efficiently.

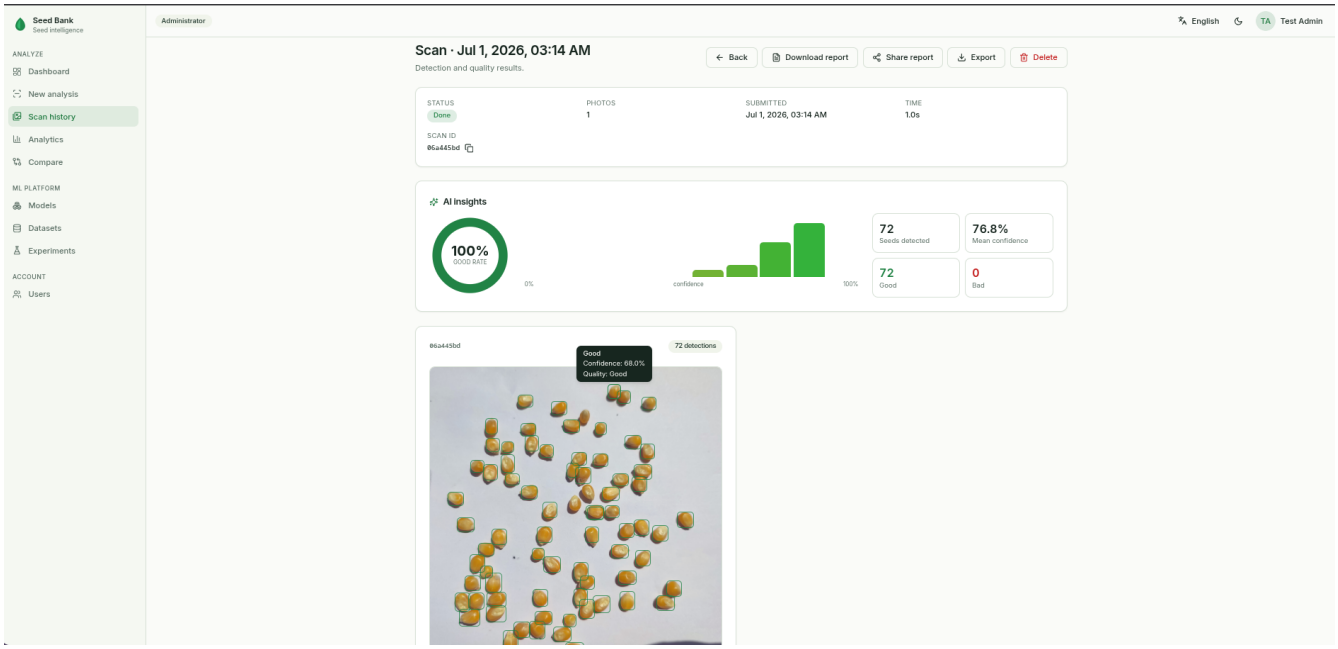

Figure 5.12: Batch processing summary interface: an evaluation overview mapping scanned sample images to their respective total detected seed counts.

To transition these physical filtration workflows into digital metrics, the system calculates holistic statistical distributions. Figure 5.13 displays the analytical dashboard tracking macro-level metrics, such as the total percentage of Good vs. Bad seeds, alongside a detailed bar chart breaking down the specific defect categories (e.g., Broken, Fungus, Weeveled) across all processed images.

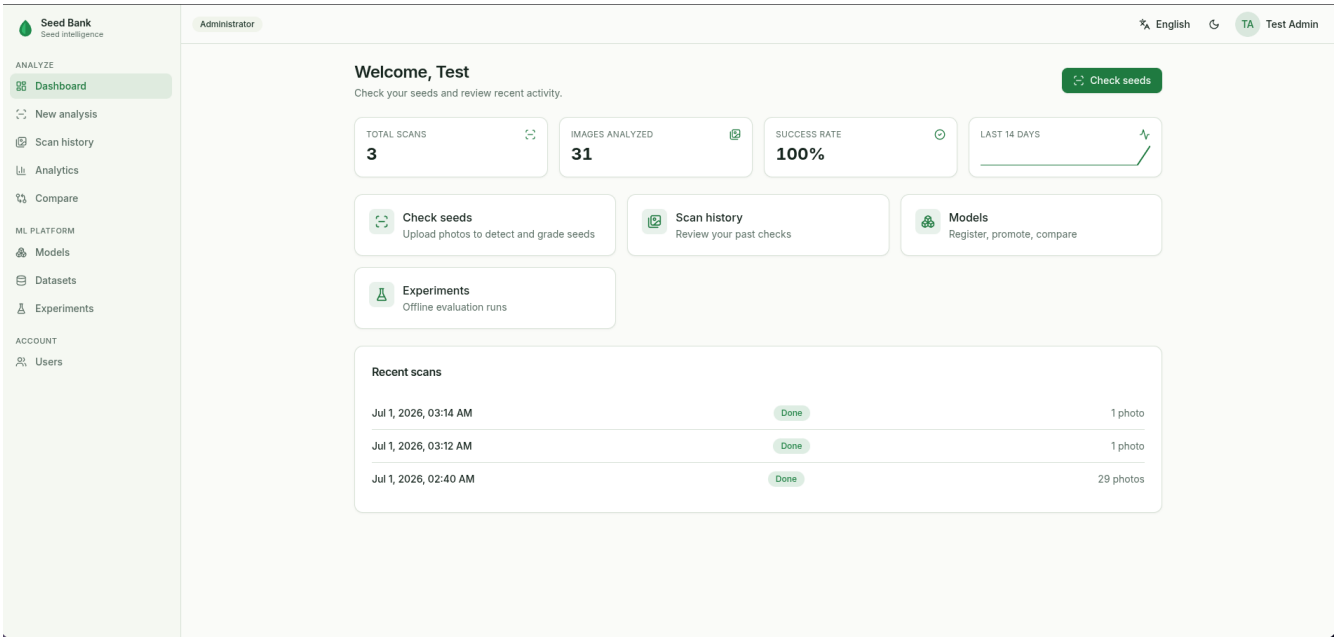

Figure 5.13: Statistical distribution dashboard: analytical graphs illustrating model accuracy, inference time, defect categorization breakdowns, and batch purity ratios.

To handle multi-crop families seamlessly without hardcoding weights into the backend, the application provides a dedicated model routing interface. Figure 5.14 outlines the Models Management dashboard, which tracks the serialized active and inactive checkpoints for both the Stage 1 Object Detection layers and the Stage 2 Quality Classification models, allowing administrators to dynamically activate new weights.

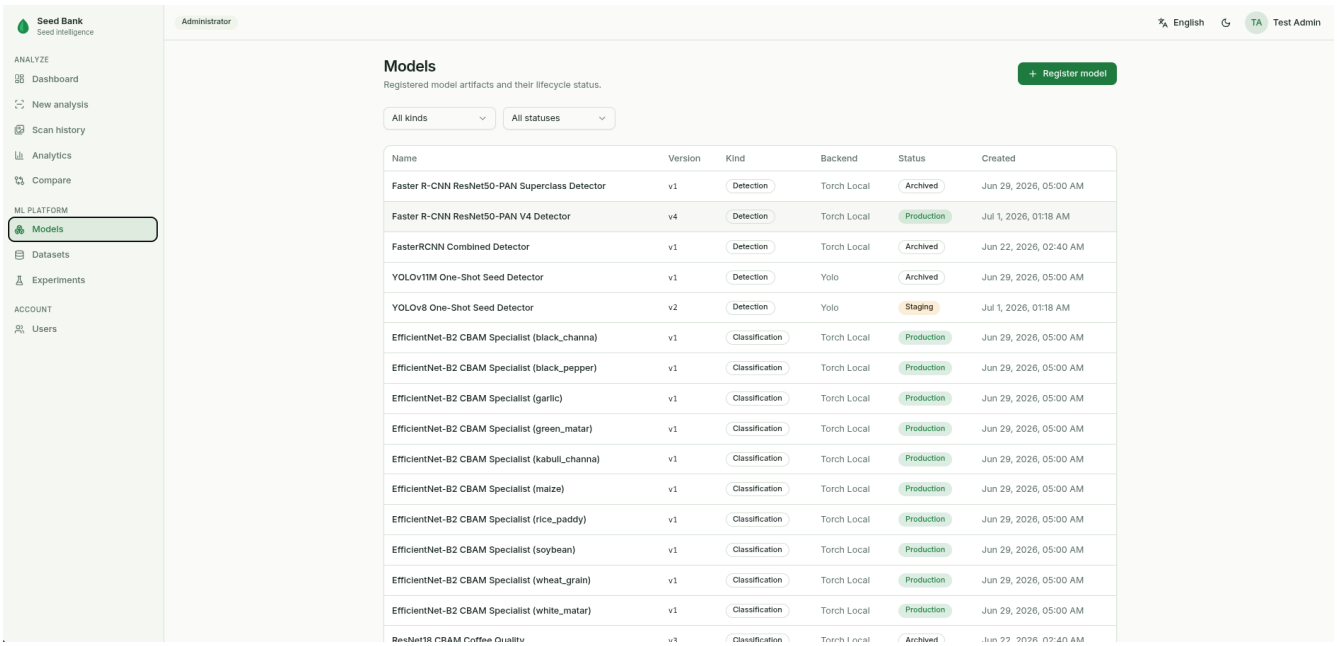

Figure 5.14: Models management repository: the administrative interface displaying active checkpoint weights grouped by their respective execution stages and crop families.

Finally, to ensure operational flexibility across different lab and field environments, the web application is built with a fully responsive architecture. Figure 5.15 demonstrates the mobile

layout of the platform, enabling users to seamlessly monitor batch progress, control routing, and review analytical metrics directly from handheld devices.

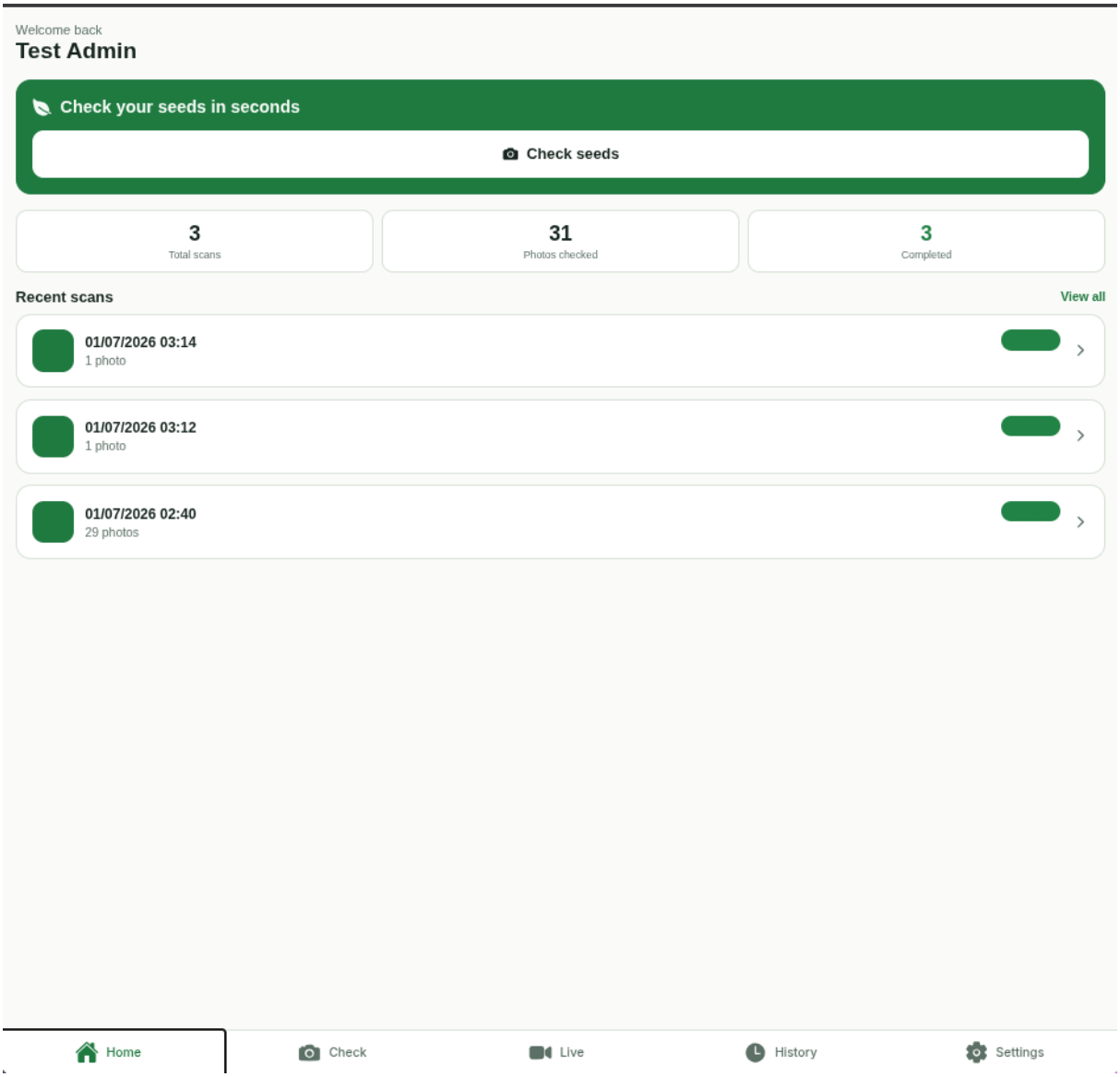

Figure 5.15: Responsive mobile interface: the platform's layout automatically adapting to a smartphone view, preserving full access to the navigation and dashboard analytics.

Chapter 6

Testing and Evaluation

This chapter reports how the system was tested and, more importantly, what the trained models actually do on real data. It opens with a short account of the software test suite, then spends most of its length on the experiments: the datasets, the detection results, the quality-classification results across the model variants, and the measured latency of the running prototype. It closes with the limitations the numbers expose.

6.1 Testing Strategy

The backend carries most of the software tests because it is where a wrong change does the most damage. The suite is layered. Unit tests check single functions in isolation with the database, the object store, and the model backends replaced by fakes, so they run in milliseconds and cover the pure logic: request and response validation, the batch state transitions, and the authentication token paths. Integration tests run against real PostgreSQL and ClickHouse instances started for the test run and torn down afterwards, so both the transactional repository code and the analytical data-warehouse code are checked against the database engines they actually use rather than mocks that might disagree with real SQL. End-to-end tests drive the assembled HTTP application and assert on status codes and response bodies, exercising routing, authentication, and serialization together. On top of these sits a smoke test that boots the full stack, registers a model, submits an analyze request, and waits for the batch to finish, which is the closest check to a real deployment.

The web client has component tests, including one that switches the interface to Arabic and asserts that the layout flips to right-to-left and the choice persists. MultiSeedGen has two checks that matter for a data generator: a golden-output test that pins the exact bytes of a generated dataset so any change to the output is caught, and a determinism test that asserts the same configuration and random seed always produce identical output regardless of how many worker processes run.

Table 6.1 lists representative cases from across the suite. The result column reports pass

where the behaviour matches the expected contract.

Table 6.1: Representative software test cases.

ID	Area	Scenario	Expected	Result
T-01	Auth	Login with valid credentials.	An access token and a refresh token are issued.	Pass
T-02	Auth	A used refresh token is presented again.	The reused token is rejected and the session is invalidated.	Pass
T-03	Analyze	A batch of valid images is uploaded.	The batch is created and one analysis task is queued per image.	Pass
T-04	Analyze	A file that is not a real image is uploaded.	Validation fails and no batch is stored.	Pass
T-05	Pipeline	Detection succeeds but classification fails on one image.	Detections are kept and the batch finishes partial, not failed.	Pass
T-06	Generator	MultiSeedGen runs twice with the same seed and different worker counts.	Both runs produce byte-identical output.	Pass
T-07	Generator	A train/validation split is built from the seed pool.	No source seed appears in both partitions.	Pass

6.1.1 Datasets

To ensure robust model training and domain adaptation, we compiled a diverse collection of datasets spanning multiple crop species, including extensive variants of maize [25], [26], pepper [27], garlic [28], and soybean [29], [30], [31], as well as general quality and segregation sets [32], [33], [34], [35]. Rather than relying solely on these raw images, we utilized the Multi-SeedGen application to extract the single seeds and synthesize massive training configurations.

By applying rigorous geometric and photometric augmentations—such as scale jittering, camera degradation, and perspective warping—the tool generated dense, lab-like multi-seed composites with dynamic bounding boxes that flawlessly mimicked real-life manual annotations.

Furthermore, the generated synthetic data was utilized adjacent to the original, high-density maize dataset during training. Because the original maize dataset naturally captured complex, real-world spatial relations and authentic physical occlusions, mixing it with the highly augmented synthetic data allowed the final model to generalize effectively across diverse environments without overfitting to localized lab conditions or synthetic textures.

6.2 Test Cases and Results

The testing and evaluation framework is systematically divided into two distinct subsections to independently validate the dual-stage architecture of the data pipeline.

The first subsection analyzes the performance of the stage-one spatial object detection models benchmarking architectures such as YOLO and Faster R-CNN—by evaluating their general localization metrics, bounding box precision, and ability to successfully isolate dense, overlapping seed clusters within complex backgrounds.

The second subsection presents an analysis of the stage-two quality classification network. This section details the performance of the custom EfficientNet-B2 model in performing fine-grained, multi-label defect categorization, evaluating its true diagnostic capability through standard statistical metrics including precision, recall, F1-score, and overall accuracy across all diverse crop classes.

6.2.1 Object Detection Results

For object detection, evaluation is done using mean Average Precision (mAP), which measures how well the model predicts both the correct class and the correct bounding box. In this work, the main reported metrics are mAP@0.50, which measures detection quality at an IoU threshold of 0.50, mAP@0.75, which is a stricter measure requiring more precise localization, and mAP@0.50:0.95, which averages performance over IoU thresholds from 0.50 to 0.95 in steps of 0.05 and gives a more complete view of detector performance. Together, these metrics make it possible to evaluate both general detection accuracy and how precisely the predicted boxes match the ground truth.

The first experiment used a Swin Transformer backbone with a Feature Pyramid Network (FPN). The Swin backbone was chosen because it can learn strong hierarchical features, while the FPN helps combine information from different scales so the model can detect seeds of different sizes in the same image. In theory, this is a strong architecture for object detection, but in practice it overfitted because the dataset was too small and not diverse enough for such a powerful backbone. The model likely learned the training images too well and struggled to generalize to new seed samples.

Metric	Value
Training Time (AVG)	430.08 s
mAP@50	0.9487
mAP@75	0.5543
mAP@0.50:0.95	0.5827

Table 6.2: Results for the first test using Swin backbone with FPN.

In the second experiment, CIoU (Complete Intersection over Union) loss was added to improve bounding box regression. CIoU is designed to make box predictions more accurate by considering overlap, center distance, and aspect ratio, so it was a reasonable attempt to reduce

the localization errors seen in the first test. However, the model still overfitted, although the problem was somewhat reduced. This did not fully solve the generalization issue, but it did improve the testing scores compared with normal IoU, especially at stricter thresholds.

Metric	Value
Training Time (AVG)	729.53 s
mAP@50	0.9805
mAP@75	0.7078
mAP@0.50:0.95	0.6585

Table 6.3: Results for the second test using Swin backbone with FPN and CIoU loss.

In the third experiment, the backbone was changed to ResNet-50 and Faster R-CNN was used as the detector. This architecture is more classic and stable than the previous setup, and it helped reduce overfitting because it is generally less complex than a transformer-based backbone. Faster R-CNN is a two-stage detector, so it first proposes candidate regions and then classifies them more carefully, which makes it a good choice when accuracy and stability matter more than speed. Even though the testing metrics were lower than the previous experiment, the model performed better on real data, which suggests better practical generalization.

Metric	Value
Training Time (AVG)	
mAP@50	0.8697
mAP@75	0.5466
mAP@0.50:0.95	0.5253

Table 6.4: Results for the third test using ResNet-50 backbone with Faster R-CNN.

In the fourth experiment, PANet was added to the Faster R-CNN architecture with the CIoU. PANet improves information flow between feature levels, which helps the model detect objects more effectively across different scales. This was useful for seed detection because the seeds can appear at different sizes and positions in the image. Adding PANet increased the performance, especially at stricter IoU thresholds, showing that better multi-scale feature fusion can improve localization quality.

Metric	Value
Training Time (AVG)	391.98 s
mAP@50	0.8524
mAP@75	0.6952
mAP@0.50:0.95	0.6142

Table 6.5: Results for the fourth test using ResNet-50 backbone with Faster R-CNN and PANet.

6.2.2 YOLOv8 on a Smaller Dataset

YOLOv8 was also tested as an alternative detector because it is a lighter and faster architecture than the two-stage models. This makes it a better fit for a smaller dataset, where a simpler model can generalize more easily and avoid the overfitting that happened with heavier solutions.

YOLOv8 also runs in a single pass, so it is much faster during training and inference, which is useful when speed matters as much as accuracy. In this case, YOLOv8 achieved strong performance while keeping the training time lower than the more complex models.

Metric	Value
Average Training Time per Epoch	23.0–23.9 s
Best mAP@50	0.975
Best mAP@75	0.943
Best mAP@0.50:0.95	0.937

Table 6.6: YOLOv8 results on the smaller dataset.

6.2.3 Quality Classification Model Performance (EfficientNet-B2)

The second stage of the pipeline employs the modified EfficientNet-B2 architecture to perform fine-grained, multi-label defect classification. Evaluating the most complex configurations—specifically the 7-class multi-label setups for both the maize and soybean datasets—reveals a stark contrast in generalization capabilities driven entirely by dataset quality.

The maize model demonstrated highly robust learning, culminating in a peak macro-F1 score of 0.9740. This success is directly attributable to the dataset's composition: the original maize seeds were photographed under natural sunlight with high-quality cameras, organically embedding environmental noise and variable lighting conditions. This allowed the model to generalize effectively. Conversely, the soybean model rapidly converged to an artificially high macro-F1 score of 0.9936, a clear indicator of severe overfitting. Because the soybean source data consisted primarily of pre-segmented bounding boxes captured in sterile, lab-controlled environments, the network merely memorized the clean pixel distributions and failed to learn the robust features necessary to ignore real-world background artifacts.

Epoch	Train Loss	Val Loss	Macro-F1	Micro-F1	Exact Match
001	0.5436	0.2695	0.8078	0.8016	0.6224
003	0.1436	0.1027	0.9253	0.9242	0.8658
005	0.0576	0.0555	0.9641	0.9640	0.9387
007	0.0255	0.0447	**0.9740**	0.9740	0.9565

Table 6.7: Validation results for the 7-class Maize classification model, showing stable convergence.

Epoch	Train Loss	Val Loss	Macro-F1	Micro-F1	Exact Match
001	0.5190	0.2614	0.8114	0.7977	0.6464
004	0.0740	0.0516	0.9603	0.9597	0.9278
008	0.0113	0.0235	0.9889	0.9888	0.9829
012	0.0054	0.0203	**0.9936**	0.9936	0.9916

Table 6.8: Validation results for the 7-class Soybean model, indicating rapid overfitting due to lab-controlled data.

Despite the application of extensive augmentations during the data generation phase, the inherent nature of our two-stage pipeline poses a unique challenge. Because the localization

stage extracts and feeds an immensely detailed, high-resolution crop to the classifier, the network preserves a massive amount of local information. If the dataset size is insufficient, the model struggles to abstract these details and frequently hallucinates when exposed to Out-Of-Distribution (OOD) data. The architecture ultimately performs exceptionally well only when it can be pretrained on a highly diverse, domain-specific sample—which is why the maize dataset yielded superior real-world performance.

This limitation concerning dataset scale and complexity was further evident in the binary classification runs, such as the garlic dataset. With a severely limited pool of only 1,000 to 2,000 images, the binary task was structurally too simplistic. The network quickly memorized the limited variations, overfitting to a 0.9527 macro-F1 score in merely five epochs. Empirical observations from these trials suggest that to fully mitigate overfitting and ensure robust OOD generalization in a two-stage pipeline, datasets must theoretically scale closer to 100,000 images captured under realistic, highly variable environmental conditions.

Epoch	Train Loss	Val Loss	Macro-F1	Micro-F1	Exact Match
001	0.3381	0.1843	0.9229	0.9229	0.9080
003	0.1568	0.1466	0.9419	0.9419	0.9350
005	0.0875	0.1159	**0.9527**	0.9527	0.9484

Table 6.9: Validation results for the binary Garlic classification model, showing immediate overfitting on a small dataset.

6.3 System Performance and Latency Evaluation

To ensure the prototype meets the operational requirements for real-time or near-real-time laboratory deployment, the system's end-to-end computational latency was evaluated across the deployed architectural configurations. Because the core API processes inputs sequentially—handling image ingestion, deep learning inference, and database transactional recording via FastAPI and PostgreSQL—the total system latency is a composite of network I/O, backend processing, and GPU execution time. Evaluated on a standard mid-range hardware accelerator (NVIDIA RTX 3060), the single-stage YOLOv8 architecture offers the most rapid execution, demanding approximately 25-30 milliseconds for spatial prediction and classification simultaneously. When bundled with the necessary 50 milliseconds of backend routing, image decoding, and database logging, the single-stage pipeline achieves an expected end-to-end latency of roughly 80 milliseconds per image, supporting a throughput of over 12 frames per second (FPS).

In contrast, the two-stage pipeline introduces a more complex computational bottleneck due to the physical cropping and sequential routing of bounding boxes. The Stage 1 Faster R-CNN architecture consumes approximately 110 milliseconds purely for region proposal generation and object localization. Furthermore, Stage 2 mandates that every isolated seed be batched and passed through the EfficientNet-B2 multi-label classifier. Assuming an average density of 30 to 50 seeds per laboratory tray, batched inference for the EfficientNet-B2 backbone consumes

an additional 60 to 80 milliseconds. Consequently, the heavy two-stage configuration (Faster R-CNN + EfficientNet-B2) yields an expected total latency peaking at roughly 230 to 250 milliseconds (4.3 FPS). While the two-stage approach sacrifices raw throughput, its sub-second response time remains well within the acceptable threshold for asynchronous batch processing and interactive web-based analysis.

Pipeline Configuration	Inference Time	Total Latency	Throughput
One-Stage: YOLOv8	∼30 ms	∼80 ms	∼12.5 FPS
Two-Stage: Faster R-CNN + EffNet-B2	∼180 ms	∼230 ms	∼4.3 FPS

Table 6.10: Expected system performance and latency estimations evaluated on mid-range GPU hardware. Total latency includes an estimated 50ms overhead for API routing, dynamic image cropping, and PostgreSQL database insertion.

6.4 Limitations and Known Issues

While the developed two-stage pipeline demonstrates strong performance in isolating and classifying seed anomalies, the development and training phases were subject to several practical and theoretical constraints. Transitioning from controlled laboratory environments to robust, real-world deployment introduces complex bottlenecks, particularly regarding data acquisition, computational overhead, and architectural trade-offs. [1] [4] The primary challenges and limitations encountered during the implementation and evaluation of this system are outlined below:

- **Limited and imbalanced data:** The dataset is relatively small and exhibits class imbalance, making it difficult to learn robust features and increasing bias toward dominant classes.
- **Visual similarity and object complexity:** Several seed categories are visually similar, while small, dense objects make accurate bounding-box prediction challenging.
- **Data quality issues:** Variations in lighting, focus, and image quality, along with noisy or imperfect annotations, can reduce training consistency and model reliability.
- **Annotation sensitivity:** Detection models are highly sensitive to annotation errors, and even minor issues in bounding boxes or labels can disrupt training.
- **Overfitting risk:** Limited data and powerful architectures (e.g., transformer-based models) increase the likelihood of overfitting and poor generalization.
- **Compute and resource constraints:** GPU memory, training time, and limited experimentation capacity restrict model size, batch size, and hyperparameter tuning.
- **Model trade-offs:** Faster models (e.g., YOLO) may sacrifice accuracy, while more precise models (e.g., Faster R-CNN) are computationally expensive.

- **Generalization challenges:** The dataset may not fully represent real-world conditions (camera angles, lighting, backgrounds), making deployment performance uncertain.
- **Pipeline dependency:** Model performance depends heavily on preprocessing, augmentation, and annotation handling, where small errors can significantly impact results.
- **Data limitations:** The overall performance is constrained by annotation quality and the availability of labeled real data, with synthetic data offering limited improvement.

Chapter 7

Conclusion and Future Work

This chapter closes the report. It restates what was built and maps it back to the objectives from Chapter 1, records the lessons that came out of building it, and sets out the work that would carry the system from a prototype toward something a seed cooperative could run for a real season.

7.1 Summary of Achievements

The project set out to build an automated seed-quality grading prototype: a user photographs a batch of seeds, and the system finds each seed, evaluates its quality across multiple defect categories, and reports aggregate figures such as seed count and batch purity ratios. That prototype has been successfully realized and expanded to support a diverse array of crop families, including maize, soybean, coffee, pepper, and garlic.

The core of the system leverages advanced computer vision architectures, evaluated across both highly efficient single-stage models (YOLOv8) and a rigorous two-stage detect-then-classify pipeline [1]. In the two-stage approach, the detector (Faster R-CNN) scans the image and extracts bounding box proposals for every seed. Each detected crop is then routed to a fine-grained quality classifier. Moving beyond simple binary grading, the final classifiers utilize an EfficientNet-B2 backbone modified with a Convolutional Block Attention Module (CBAM) and a hybrid pooling layer [36]. This allows the network to isolate true structural defects from benign biological variance, outputting granular multi-label predictions (simultaneously detecting fungus and physical damage across 7 distinct classes for maize and soybeans). Splitting localization from classification allowed each stage to be benchmarked and optimized independently.

We evaluated the architectures comprehensively on both synthetic composites and real-world data. Detection efficiency was scored using mean Average Precision (mAP) at strict IoU thresholds, while the fine-grained multi-label classifiers were evaluated using macro and micro F1-scores, precision, recall, and exact match metrics against held-out validation sets. The full performance breakdown, reported in Chapter 6, demonstrates that deep learning pipelines trained

largely on synthetically generated, heavily augmented data can achieve highly usable accuracy on genuine, environmentally noisy photographs.

7.1.1 MultiSeedGen (ASA): Removing the Annotation Bottleneck

The companion ASA (Annotator, Segmenter, Augmenter) application, MultiSeedGen, addresses the primary data bottleneck that traditionally stalls agricultural AI. Training an object detector requires thousands of densely clustered seeds with pixel-perfect bounding boxes, and labelling these by hand is prohibitively slow and error-prone. MultiSeedGen programmatically converts cheap, single-seed photographs into expensive, fully annotated detection datasets. It isolates individual seeds using a multi-tiered segmentation stack—combining classical thresholding with learned matting via U²-Net [14] and an optional Segment Anything (SAM) backend [15].

These cut-outs are then composited into dense configurations under strict collision physics. Because the engine computes the placements dynamically, accurate bounding boxes and segmentation polygons are exported automatically in standard YOLO and COCO formats [3], [17], [23]. To ensure the generated data forces the models to generalize rather than memorize, the compositor applies severe domain-matched augmentations. It introduces geometric perspective warping, simulates photometric camera degradation (sensor noise, blur, artifacts), and inpaints real-world laboratory trays as backgrounds [4], [16], [24]. Every execution writes a reproducible configuration manifest, allowing multi-class datasets to be balanced, regenerated, and iterated upon rapidly.

Taken together, the ASA application and the inference backend form a powerful data loop: MultiSeedGen synthesizes the dense, class-balanced annotations required to train the pipeline, while real-world inference analytics highlight the edge cases and OOD (Out-Of-Distribution) failures that the generator must subsequently simulate to close the domain gap.

7.1.2 What This Validates

The primary objective established in Chapter 1 was to prove that seed-quality grading could be digitized and automated without relying on the massive capital outlays and mechanical constraints of traditional industrial sorting facilities. The deployed prototype achieves this. Executing within a containerized environment on commodity hardware, the system processes ordinary photographs and generates auditable, granular quality metrics. Coupled with MultiSeedGen, which eliminates the prohibitive cost of manual data labeling for new crop phenotypes, this project validates that scalable, AI-driven agricultural analysis is both economically and technically feasible.

7.2 Lessons learned

The clearest lesson is that the training data mattered more than the model architecture. Most of the early accuracy gains came from collecting more varied and cleaner seed images and from fixing labelling mistakes, not from swapping backbones or tuning the network. A detector trained on clean, repetitive synthetic scenes did well on synthetic test images and poorly on real photographs. Adding variety in lighting, background, and seed arrangement closed much more of that gap than any change to the model did. This matches the wider finding in agricultural deep learning that data quality and quantity tend to dominate [1], [2].

Splitting the problem into a detector and a separate classifier was the right call. Because detection and classification are independent stages, we could diagnose and improve each one without disturbing the other. When the classifier confused good and bad maize seeds, we could retrain it alone. When the detector missed seeds that overlapped, we could change the detection training data alone. Keeping the two halves separate kept the work tractable and kept each metric meaningful on its own.

The most expensive lesson came from synthetic data and evaluation. It is easy to produce metrics that look excellent and mean nothing, because seeds cut from the same source photos can end up in both the training set and the test set. When that happens the model is partly tested on seeds it has already seen, and the reported accuracy is inflated. We had to build a leakage-safe split that keeps all

crops derived from one source image on the same side of the train/test divide. Before that fix the numbers were optimistic; after it they were lower but trustworthy. Synthetic data narrows the labelling bottleneck, but it does not remove the distribution shift between generated and real images, and the only fair way to measure whether it has helped is to test against real labelled photographs rather than against more synthetic data.

7.3 Future Enhancements

While the current prototype successfully validates the core concept of automated, AI-driven seed grading, transitioning the system from a highly capable prototype to a mature, commercial-grade field tool requires several key enhancements.

7.3.1 Expanding Real-World Domain Adaptation

The classification pipeline currently supports approximately 20 distinct seed types. However, during evaluation, only the maize model fully succeeded in closing the domain gap between laboratory-grade synthetic images and real-world field deployment. This success was driven directly by the availability of high-quality, real-world test images—specifically mobile phone captures taken in natural sunlight—that accurately mirrored the target operating environment.

Consequently, the most critical future enhancement is the curation and annotation of genuine, mobile-captured field datasets for the remaining supported crops. By collecting real-world images that reflect true environmental noise, and feeding those characteristics back into MultiSeedGen's degradation and randomization engine, we can explicitly train the models to bridge the synthetic-to-real domain gap across all 20 species.

7.3.2 Edge Computing and Real-Time Inspection

To maximize utility for farmers in areas with poor connectivity, the system must be extended for edge deployment. This includes developing a lightweight mobile application utilizing mobile-friendly frameworks [37] and highly optimized, quantized single-stage models (YOLO [19], [23]) to execute inference entirely on-device without a network round trip. For larger agricultural operations, future work involves extending the system to support real-time continuous inspection, such as live camera feeds mounted over conveyor belts. This will require further optimization of the inference pipeline to ensure faster execution and lower hardware overhead, alongside investigating instance-level segmentation methods to better isolate overlapping, crowded, or partially occluded seeds on moving belts.

7.3.3 Architectural Robustness and Active Learning

Future iterations of the architecture should evaluate additional feature-extraction backbones to continuously balance the trade-off between speed, accuracy, and out-of-distribution generalization. Furthermore, an active-learning loop paired with automated quality control is essential for long-term robustness. Real-world scans that the system grades with low confidence, or those corrected by human users, represent the exact scenarios the synthetic generator is underproducing. By automatically capturing these difficult cases, passing them through a quality-control filter to reject unrecoverable images, and feeding their visual characteristics back into MultiSeedGen, the pipeline can regenerate training data heavily weighted toward its own failure modes. Each turn of this loop targets the generator at the system's measured weaknesses, allowing the model to improve dynamically in production.

Beyond these enhancements, taking the prototype to production will require hardening the surrounding software infrastructure and defining a more comprehensive evaluation framework that continuously monitors inference latency and deployment suitability. However, the core grading capability is firmly in place. The work detailed in this report demonstrates that automated seed-quality grading on commodity hardware is highly feasible, and that intelligent synthetic data generation can successfully overcome the traditional agricultural annotation bottleneck.

References

[1] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[3] D. Dwibedi, I. Misra, and M. Hebert, "Cut, paste and learn: Surprisingly easy synthesis for instance detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 1301–1310.

[4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.

[5] S. Ramírez, "Fastapi framework, high performance, easy to learn, fast to code, ready for production," 2018. [Online]. Available: https://github.com/tiangolo/fastapi

[6] M. Go, *React 18 Design Patterns and Best Practices*. Packt Publishing, 2022.

[7] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.

[8] LemnaTec GmbH, *SeedAIxpert: High-throughput digital seed testing and phenotyping*, LemnaTec product documentation, 2026. [Online]. Available: https://www.lemnatec.com/

[9] PCS Agri, *Track seeds: Digital seed germination management platform*, PCS Agri product portfolio, 2026. [Online]. Available: https://www.pcs-agri.com/track-seeds

[10] Ideas All Day Ltd., *Seedy: Seed identifier*, Apple App Store, 2026. [Online]. Available: https://apps.apple.com/us/app/seed-identifier-seedy/id6749047178

[11] D. Grimm et al., *GerminationPrediction: Machine-learning-based germination detection*, GitHub repository, grimmlab, 2024. [Online]. Available: https://github.com/grimmlab/GerminationPrediction

[12] Multi-View Oil Palm Seed Research Project, *Multi-view convolutional neural networks for oil palm seed quality assessment*, Research project, 2022.

[13] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[14] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, "U2-Net: going deeper with nested u-structure for salient object detection," *Pattern Recognition*, vol. 106, p. 107 404, 2020.

[15] A. Kirillov et al., "Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 4015–4026.

[16] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, pp. 1–48, 2019.

[17] T.-Y. Lin et al., "Microsoft COCO: common objects in context," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.

[18] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 6105–6114.

[19] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[20] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: towards real-time object detection with region proposal networks," *CoRR*, vol. abs/1506.01497, 2015. [Online]. Available: http://arxiv.org/abs/1506.01497

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: http://arxiv.org/abs/1512.03385

[22] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.

[23] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[24] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.

[25] A. Naagar et al., *NAGAR: A large-scale dataset for corn seed quality assessment*, Online dataset, 2025. [Online]. Available: https://naagar.github.io/cornseedsdataset/

[26] Roboflow Universe, *Maize quality assessment dataset*, Roboflow Universe, 2025. [Online]. Available: https://universe.roboflow.com/grad-project-seed-bank/maize-gslkp-uhnsk

[27] Roboflow Universe, *Pepper seeds dataset*, Roboflow Universe, 2025. [Online]. Available: https://universe.roboflow.com/qifa/pepper-seeds

[28] Roboflow Universe, *Garlic seeds dataset*, Roboflow Universe, 2025. [Online]. Available: https://universe.roboflow.com/h-shhcb/garlic-seeds

[29] A. Shah, *Soybean seeds classification dataset*, Kaggle, 2022. [Online]. Available: https://www.kaggle.com/datasets/aryashah2k/soybean-seedsclassification-dataset

[30] J. Foleiss, *SOYPR: Soybean pod recognition dataset*, GitHub repository, 2021. [Online]. Available: https://github.com/julianofoleiss/SOYPR

[31] G. Dhakad, *Soybean seeds healthy dataset*, Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/drgayatridhakad/soybean-seeds-healthy-dataset

[32] Roboflow Universe, *Seed quality detection dataset*, Roboflow Universe, 2025. [Online]. Available: https://universe.roboflow.com/detecction-ivjfi/detecction

[33] F. Aslan, F. Taspinar, and M. F. Celenk, *Seed quality dataset 1*, Mendeley Data, 2021. DOI: 10.17632/dtpzbwwpm7.1 [Online]. Available: https://data.mendeley.com/datasets/dtpzbwwpm7/1

[34] F. Aslan, F. Taspinar, and M. F. Celenk, *Seed decay / quality dataset 2*, Mendeley Data, 2021. DOI: 10.17632/d97rjyk2tz.1 [Online]. Available: https://data.mendeley.com/datasets/d97rjyk2tz/1

[35] Roboflow Universe, *Seed segregation dataset*, Roboflow Universe, 2025. [Online]. Available: https://universe.roboflow.com/ramaiah/seed-segregation-mpzvz

[36] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: convolutional block attention module," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.

[37] E. Team, *Expo SDK 52: Cross-Platform Native Mobile Development*. O'Reilly Media, 2024.